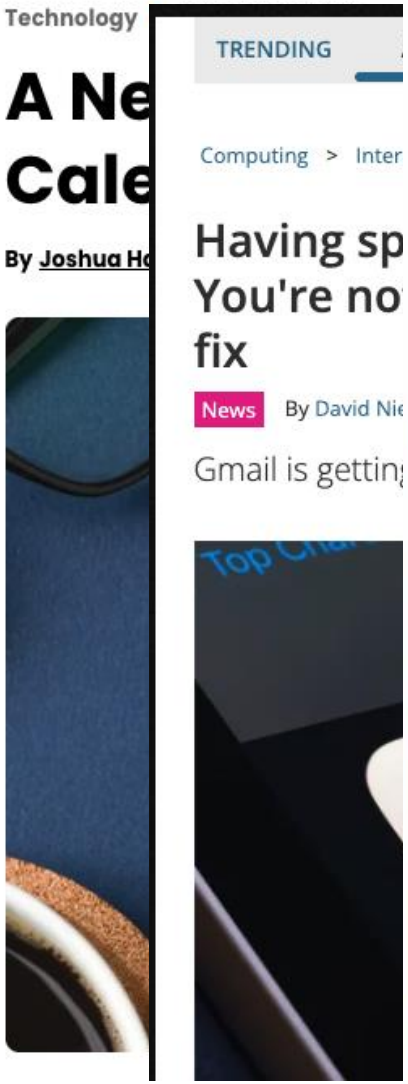
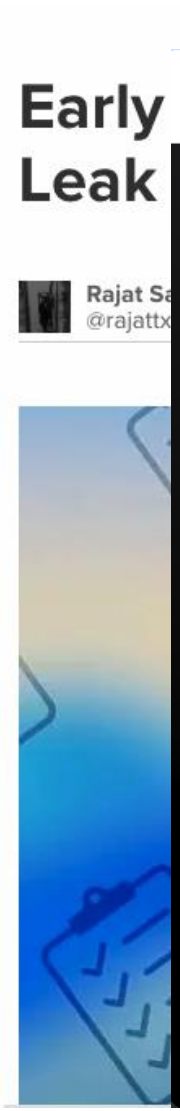
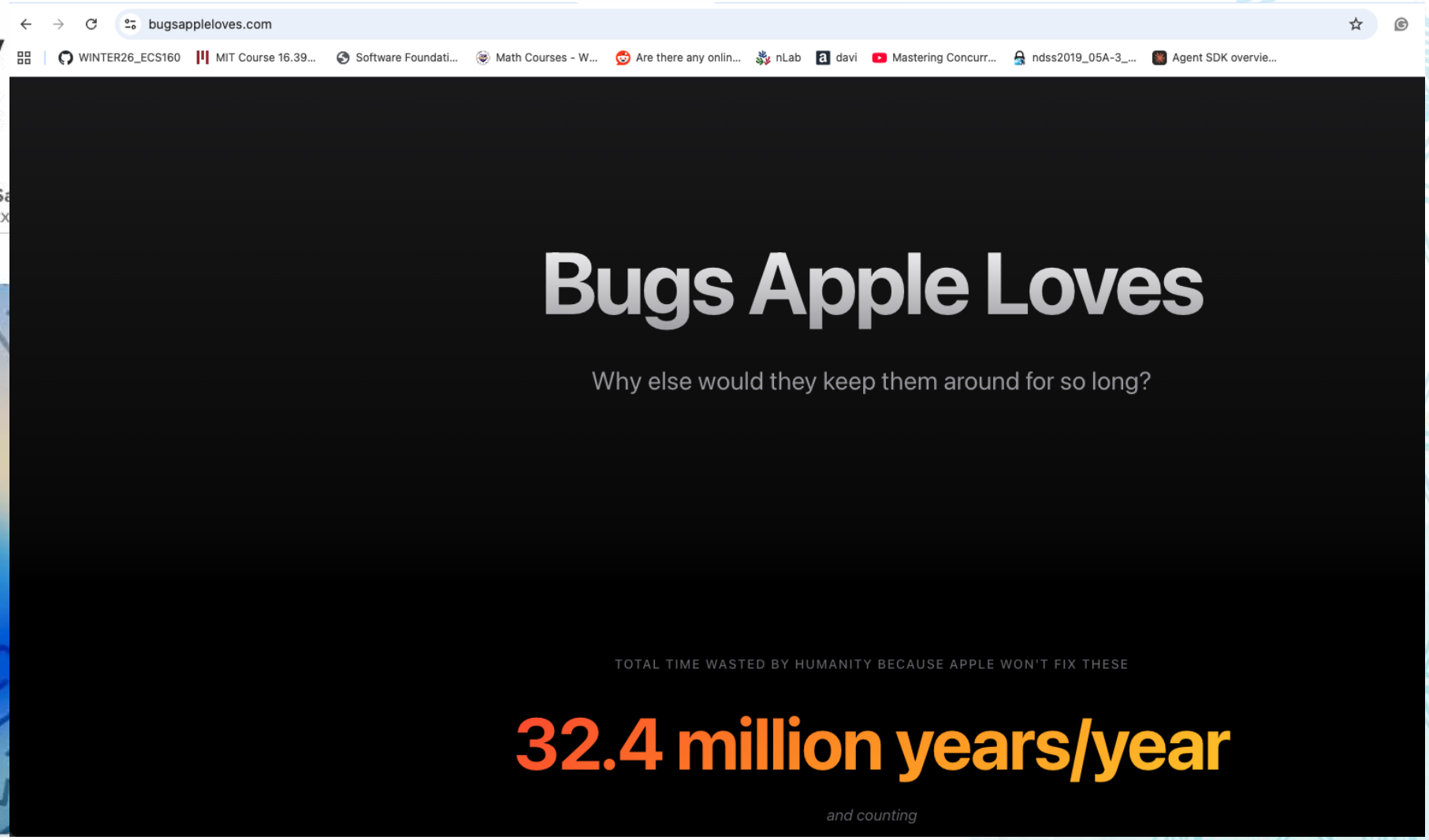
Software Testing

Tapti Palit

Software has bugs







Some bugs are security vulnerabilities

cybernews.com/security/high-severity

26_ECS160 MIT Course 16.39... Soft

Publi

Image

NGII
enab
redir

Malware

NEWS

Upda
day b
execu
webp

by Pieter Ar

Google has
in how the

CVE-2026-
it serious e
release.

Home > News > Microsoft > Microsoft patches actively exploited Office zero-day vulnerability

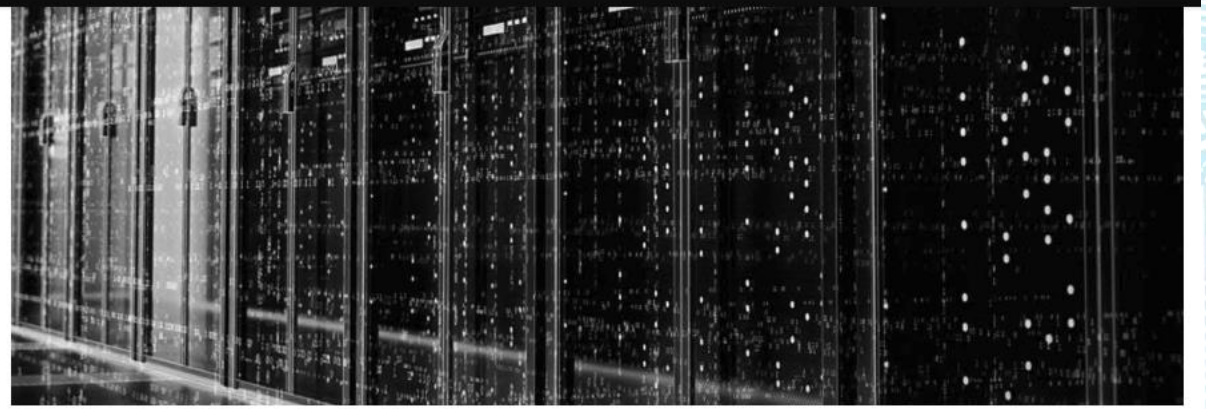
Micro

By Sergiu

Microsoft has released emergency out-of-band security updates to patch a high-severity Microsoft Office

CYBERCRIME MAGAZINE

ABOUT RESEARCH NEWS



Cybercrime Costs. PHOTO: Cybercrime Magazine.

Cybercrime To Cost The World $10.5 Trillion Annually By 2025

Special Report: Cyberwarfare In The C-Suite.

– *Steve Morgan, Editor-in-Chief*

Sausalito, Calif. – Nov. 13, 2020

If it were measured as a country, then cybercrime — which is predicted to inflict damages totaling $6 trillion USD globally in 2021 — would be the world's third-largest

Software testing

- Process of finding bugs and vulnerabilities in software
- *"Program testing can be used to show the presence of bugs, but never to show their absence"*
  - By E. W. Dijkstra

Types of software testing

- Testing based on dynamic analysis
  - Software is executed
  - Monitor for assertion failures and other bugs during execution
  - Unit testing (JUnit), integration testing, ***fuzz testing***
- Testing based on static analysis
  - Source code is analyzed without execution
  - Analysis identifies known bug patterns (uninitialized variables, unsafe data flow, null pointer dereferences)
  - ***Dataflow analysis, control flow analysis,*** abstract interpretation
- Hybrid approach: ***symbolic execution***

Dynamic Testing

Recap: unit testing with JUnit

- Test individual functions
- Add assertions to check if post-conditions hold
- Test corner-cases
- Manually created

```
public Car {
    // --snip
    public void accelerate() {
        this.speed += 10;
    }
}

public class CarTest {
    @Test
    public void testAccelerate() {
        Car car = new Car("Toyota", 50);
        car.accelerate();
        assertEquals(car.getSpeed(), 60);
    }
}
```

Hard to cover all code with unit testing

- Linux kernel
  - ~30 million lines of code
- Google's chromium browser
  - ~28 million lines of code
- Hard to manually write unit tests to cover all this code
  - ***Need automated testing***
- Automated testing is a sub-field of automated software engineering

Silent failures

- Some bugs lead to crashes, assertion failures
  - Detecting crash bugs requires exercising the buggy code with the right inputs
  - E.g, `assert(i > 0)`
- Some bugs are silent

Silent failures and UBs

C/C++

```
#include <stdio.h>

int main(int argc, char** argv) {
    char a = ‘A’;
    char arr[4] = {‘a’, ‘b’, ‘c’,
‘d’};
    int index = atoi(argv[1]);
    printf(“%c\n”, arr[index]);

}

> gcc test.c –o test
> ./test 10
> Output?
```

Java

```
class MyApp {
    public static void main(String[] args) {
        char a = ‘A’;
        char arr[] = {‘a’, ‘b’, ‘c’, ‘d’};
        int index = Integer.parseInt(args[0]);
        System.out.println(arr[index]);
    }
}

> javac MyApp.java
> java MyApp.class 10
> # Output?
```

Silent failures and UBs

C/C++

```
#include <stdio.h>

int main(int argc, char** argv) {
    char a = ‘A’;
    char arr[4] = {‘a’, ‘b’, ‘c’,
‘d’};
    int index = atoi(argv[1]);
    printf(“%c\n”, arr[index]);

}

> gcc test.c -o test
> ./test 10
> ?
```

Java

```
class MyApp {
    public static void main(String[] args) {
        char a = ‘A’;
        char arr[] = {‘a’, ‘b’, ‘c’, ‘d’};
        int index = Integer.parseInt(args[0]);
        System.out.println(arr[index]);
    }
}

> javac MyApp.java
> java MyApp.class 10
> Exception in thread "main"
java.lang.ArrayIndexOutOfBoundsException: Index 10
out of bounds for length 4
```

Undefined Behavior (UB)

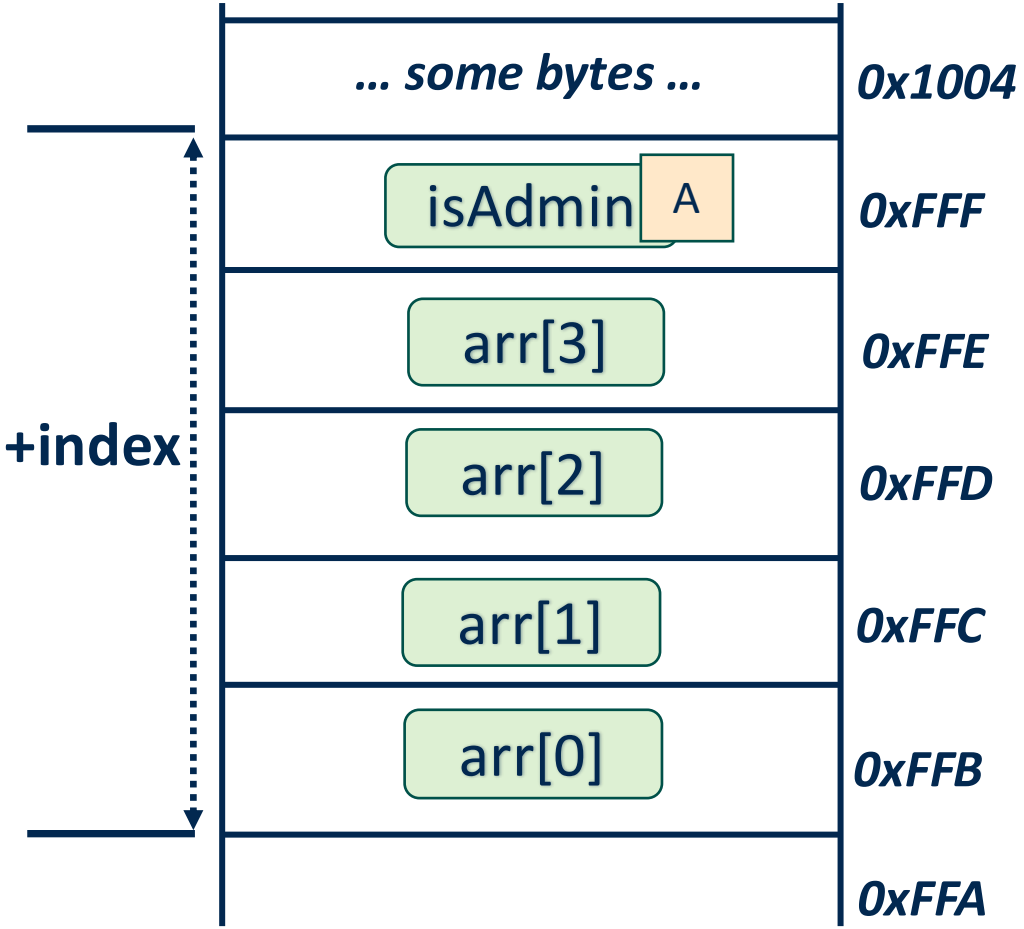
Memory safety

- Languages like C lack memory-safety
  - Can be exploited by attackers
- Buffer-overflow

```
int main(int argc, char** argv) {
    int isAdmin = 0;
    char arr[4];

    int index = atoi(argv[1]);

    arr[index] = 'A';
    // --snip
}
```

Can silently corrupt the isAdmin variable!

Memory safety




```
int main(int argc, char** argv) {
    char isAdmin = check_admin_access();
    char arr[4];

    int index = atoi(argv[1]);

    arr[index] = 'A';

    if (isAdmin) {
        delete_all_files();
    }
}
```

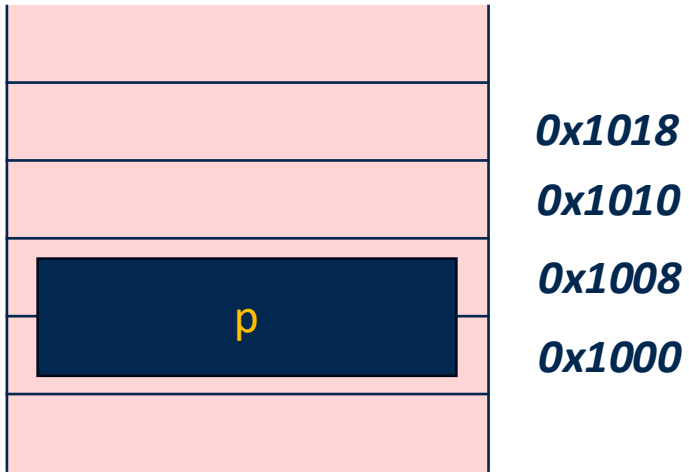
Memory safety

- Languages like C lack memory-safety
  - Can be exploited by attackers
- Buffer-overflow
- Use-after-free

```
int main(int argc, char** argv) {
    char* p = malloc(100);
    // do stuff with p
    free(p);
    // 1000 program statements
    char* passwd = malloc(20);

    p[5] = 'X';
}
```

UAF bug!

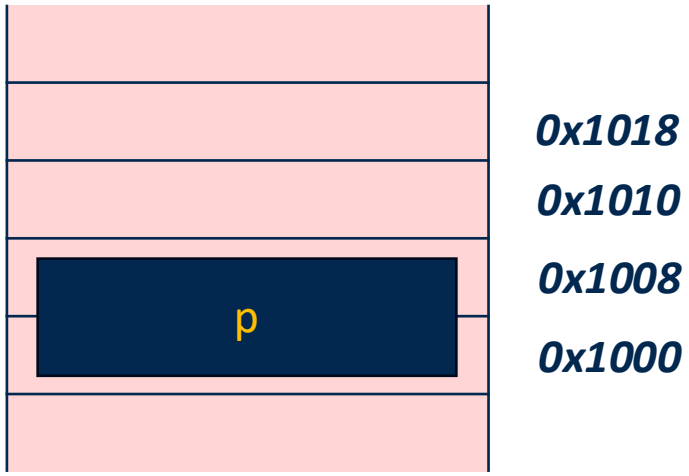
UAF bug details



Virtual Memory

```
int main(int argc, char** argv) {
    char* p = malloc(16);
    // do stuff with p
    free(p);
    // Many program statements
    char* passwd = malloc(16);


    p[5] = 'X';
}
```

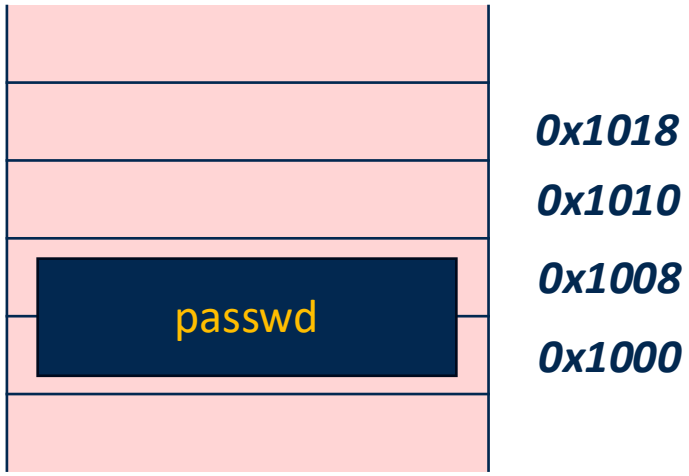
UAF bug details



Virtual Memory

```
int main(int argc, char** argv) {
    char* p = malloc(16);
    // do stuff with p
    free(p);
    // Many program statements
    char* passwd = malloc(16);


    p[5] = 'X';
}
```

UAF bug details

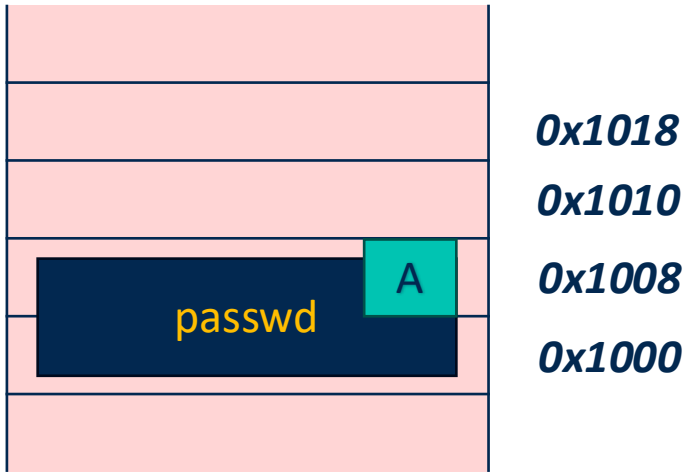


Virtual Memory

```
int main(int argc, char** argv) {
    char* p = malloc(16);
    // do stuff with p
    free(p);
    // Many program statements
    char* passwd = malloc(16);


    p[5] = 'X';
}
```

Attacker manipulates the heap to make sensitive heap objects be allocated at the site with UAF pointer accesses

UAF bug details



Virtual Memory

```
int main(int argc, char** argv) {
    char* p = malloc(16);
    // do stuff with p
    free(p);
    // Many program statements
    char* passwd = malloc(16);

    p[i] = 'X';
}
```

UAF write corrupts the sensitive data object

Memory safety

- Languages like C lack memory-safety
  - Can be exploited by attackers
- Buffer-overflow
- Use-after-free
- Double-free
- None of these bugs necessarily crash the program!

```
int main(int argc, char** argv) {
    char* p = malloc(100);
    // do stuff with p
    free(p);

    char* passwd = malloc(20);

    free(p);
}
```

Crash and silent failures

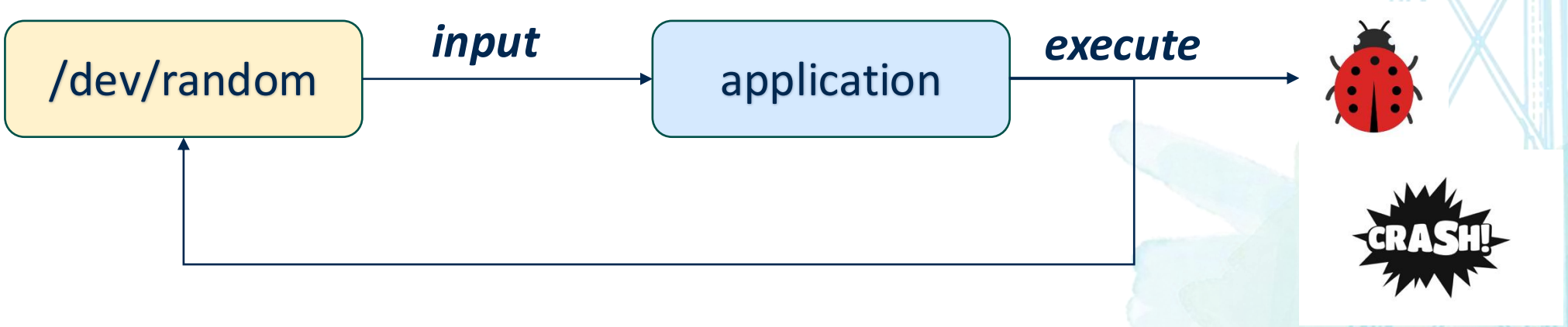
- Some bugs lead to crashes, assertion failures
  - Detecting crash bugs requires exercising the buggy code with the right inputs
  - E.g, `assert(i > 0)`
- Some bugs are silent
  - Detecting silent (UB) bugs requires both, exercising the buggy code with the right inputs ***AND an oracle to identify the buggy execution***

Fuzz testing

- Goal: to find application inputs that reveal bugs
- Test the entire application instead of individual functions
- Generate inputs randomly until an error is uncovered
  - ***How to generate new inputs?***

Fuzz testing

Source for random input on *nix systems



Black-box testing

Types of bugs

- Incorrect argument validation, incorrect type casting
- Buffer overflows, memory leak, div-by-zero, use-after-free, segmentation fault, and so on…
- Logic errors through assertion failures
- Applications written in any language can be fuzzed

Blackbox fuzzing design

- Simple loop invoking app with new inputs
- No internal visibility into the app

```
int main(int argc, char* argv[]) {
  int sum = process_file(argv[1]);
  printf("%d\n", sum);
}
```

```
int process_file(const char *p){
  FILE *f=fopen(p,"r"); if(!f) return -1;
  char s[256]; int sum=0;
  while(fgets(s,sizeof s,f)){
    if(s[0]=='#'||s[0]=='\n') continue;
    else if(s[0]=='A') sum+=1;
    else if(s[0]=='B') sum*=2;
    else if(s[0]=='C' && s[1]=='!') { fclose(f);
return -2; }
  }
  fclose(f); return sum;
}
```

Fuzzing target

Blackbox fuzzing design

- Simple loop invoking app with new inputs
- No internal visibility into the app
- E.g., simple blackbox fuzzing loop

```
int main(int argc, char* argv[]) {
  char* random_bytes = read("/dev/random");
  write("/tmp/f.dat", random_bytes);
  process_file("/tmp/f.dat");
}

int process_file(const char *p){
  FILE *f=fopen(p,"r"); if(!f) return -1;
  char s[256]; int sum=0;
  while(fgets(s,sizeof s,f)){
    if(s[0]=='#'||s[0]=='\n') continue;
    else if(s[0]=='A') sum+=1;
    else if(s[0]=='B') sum*=2;
    else if(s[0]=='C' && s[1]=='!') { fclose(f);
return -2; }
  }
  fclose(f); return sum;
}
```

Fuzzing target

Pros and cons

- Pros: generating random inputs is simple
- Cons: random inputs are not good inputs
- E.g., fuzzing a SQL library with SQL commands
  - Real inputs: `CREATE TABLE mytable AS (id INTEGER, … );`
  - Random inputs: `aVkjW3txpLZ044zo5kLdUlsU2MtlLNkhwxI8Aew7c0KPfTS115i2rzfBlgod`
  - The database will reject such statements as invalid SQL and not execute the SQL command -> the SQL code will not even be tested

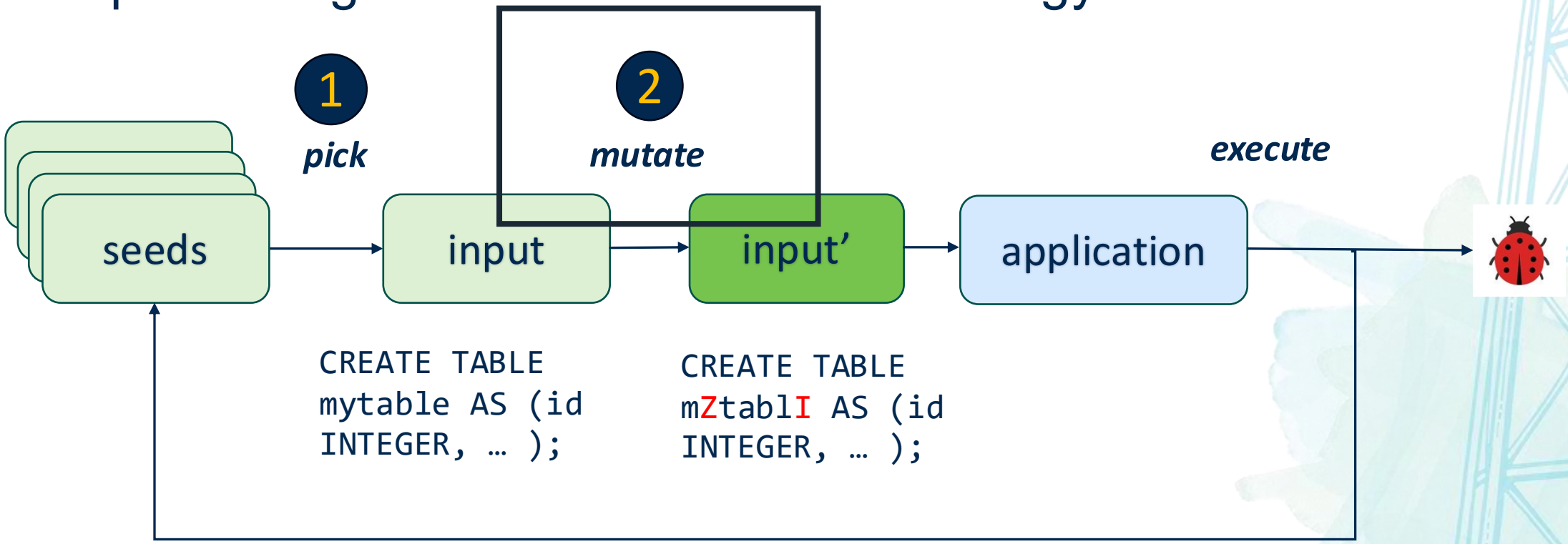
Pros and cons

- Real inputs: `CREATE TABLE mytable AS (id INTEGER, … );`
- Random inputs: `aVkjW3txpLZ044zo5kLdUlsU2MtlLNkhwxI8Aew7c0KPfTS115i2rzfBlgod`
- The database will reject such statements as invalid SQL and not execute the SQL command -> the SQL code will not even be tested

```
int process_sql(const char *query) {
    if (!check_if_sql_valid(query))
        return ERROR_INVALID;

    // Deep logic never reached by
    // most mutated inputs
    execute_query(query);
}
```

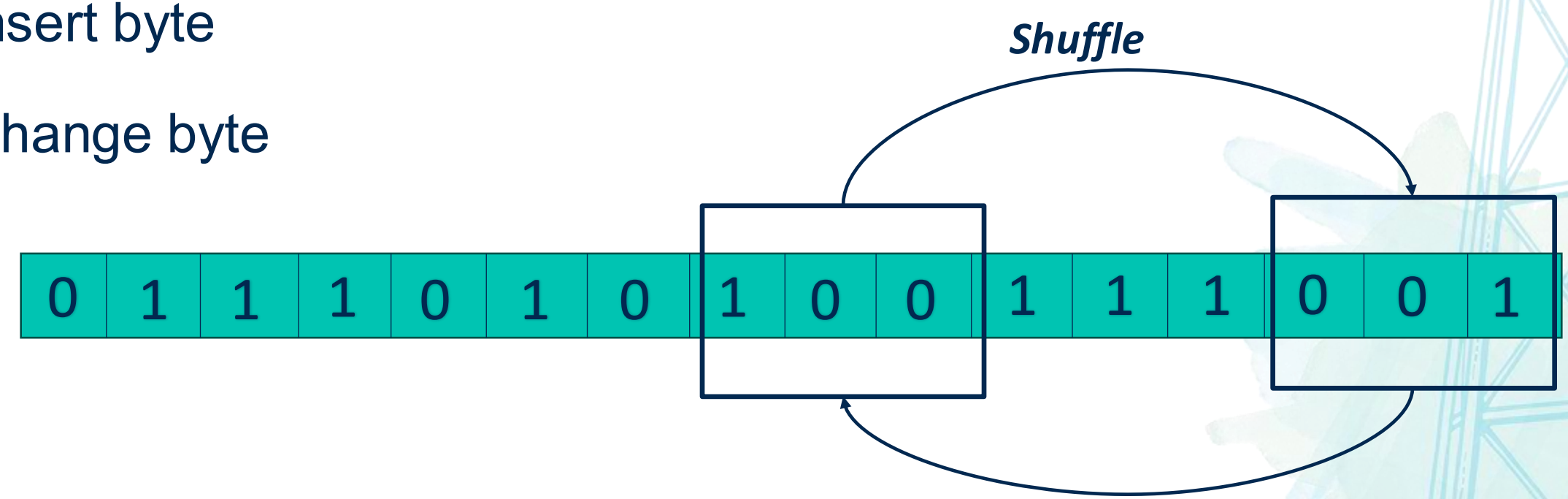
Mutation fuzzer - generate inputs via mutation

- Start with some known valid inputs called *"seeds"*
- Keep mutating them based on some "strategy"



Mutation strategy – byte-level transformations

- Shuffle bytes
- Erase bytes
- Insert byte
- Change byte



Mutation strategy – byte-level transformations

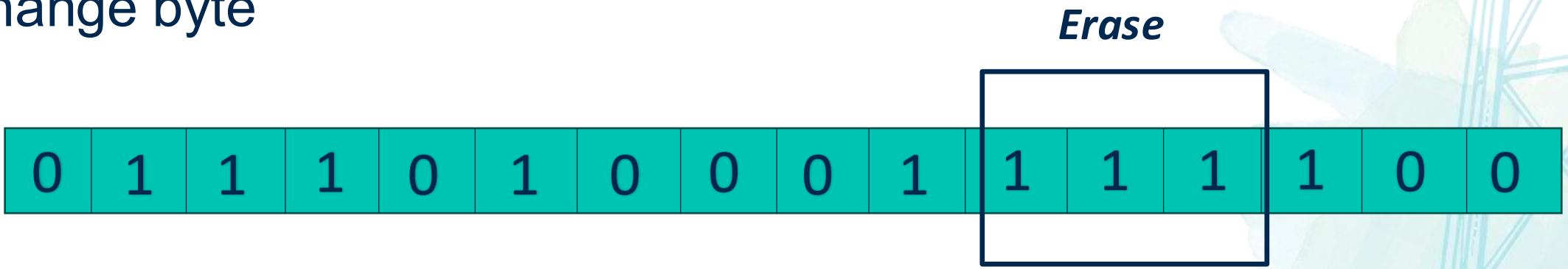
- Shuffle bytes
- Erase bytes
- Insert byte
- Change byte

New input

0	1	1	1	0	1	0	0	0	1	1	1	1	1	0	0

Mutation strategy – byte-level transformations

- Shuffle bytes
- Erase bytes
- Insert byte
- Change byte



Mutation strategy – byte-level transformations

- Shuffle bytes
- Erase bytes
- Insert byte
- Change byte

Next input

0	1	1	1	0	1	0	0	0	1	1	0	0

Mutation strategy – byte-level transformations

- Benefits:
  - Simple to implement
  - Good for binary inputs and streams
    - JPEG file encoder/decoder
- Disadvantages:
  - Generates many invalid inputs for applications that expect structured inputs

Mutation strategy – dictionaries

- Programmer provides a dictionary of “keywords”
  - For SQL, could include `CREATE`, `DELETE`, `INSERT`, `INTO`, `FROM`, `INTEGER`, and so on…
  - For HTML, could include HTML tags `<html>`, `<body>`, `<p>`, and so on…
- Mutate the input by replacing or adding a chosen keyword from this dictionary
- Input - `<html><body><p>Hello world </p></body></html>`
  Dictionary - `{<html>, <body>, <p>,…}`
  Mutated input –
  `<html><body><p> Hello world <p>, Welcome!</p> </p></html>`

Mutation strategy – dictionaries

- Benefits:
  - Still simple to implement
  - Works reasonably well for structured inputs
- Disadvantages
  - Still generates many invalid inputs for applications that expect structured inputs

Grammar-aware fuzzing

- Previous mutators generate many syntactically invalid inputs
- For structured formats (SQL, HTML, JSON), the parser rejects them before reaching deeper application logic
- Wasted fuzzing effort
  - Need a way to guarantee valid input structure

```
int process_sql(const char *query) {
    if (!check_if_sql_valid(query))
        return ERROR_INVALID;

    // Deep logic never reached by
    // most mutated inputs
    execute_query(query);
}
```

Grammar-aware fuzzing

- Define the input format as a context-free grammar (BNF rules)
- Generate inputs by randomly expanding production rules from the start symbol
- Every generated input is syntactically valid *by construction*

Grammar-aware fuzzing

- Define the input format as a context-free grammar (BNF rules)
- Generate inputs by randomly expanding production rules from the start symbol
- Every generated input is syntactically valid *by construction*

Context free grammar for simplified SQL

- A CFG is grammar that describes a language
- Production rules dictate how a valid statement can be produced
- Grammar-based fuzzing used these production rules to generate valid statements

```
<query>   ::= SELECT <columns> FROM <table>
            | SELECT <columns> FROM <table>
               WHERE <cond>
<columns> ::= <col> | <col>, <columns>
<col>     ::= [A-Za-z]+
<table>   ::= [A-Za-z]+
<cond>    ::= <col> = <value>
<value>   ::= [0-9]+ | '[A-Za-z]+'
```

Context free grammar for simplified SQL

- A CFG is grammar that describes a language
- Production rules dictate how a valid statement can be produced
- Grammar-based fuzzing used these production rules to generate valid statements

```
<query>     ::= SELECT <columns> FROM <table>
              | SELECT <columns> FROM <table>
                WHERE <cond>
<columns>   ::= <col> | <col>, <columns>
<col>       ::= [A-Za-z]+
<table>     ::= [A-Za-z]+
<cond>      ::= <col> = <value>
<value>     ::= [0-9]+ | '[A-Za-z]+'
```

Context free grammar for simplified SQL

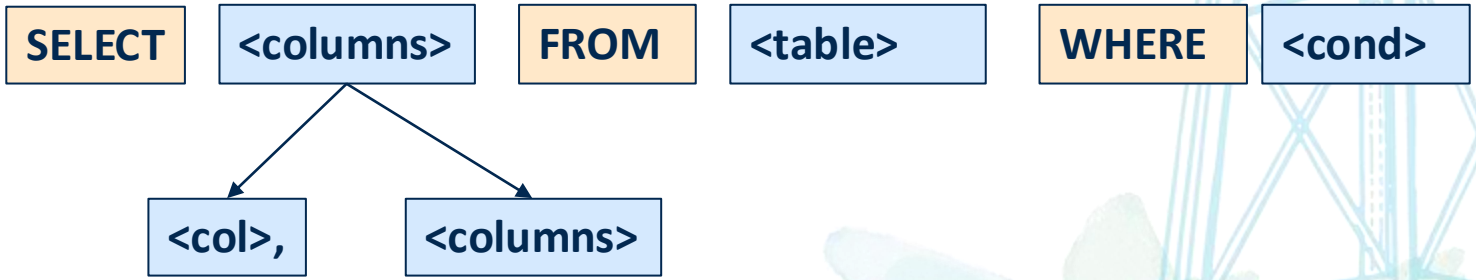
- A CFG is grammar that describes a language
- Production rules dictate how a valid statement can be produced
- Grammar-based fuzzing used these production rules to generate valid statements

```
<query>   ::= SELECT <columns> FROM <table>
            | SELECT <columns> FROM <table>
              WHERE <cond>
<columns> ::= <col> | <col>, <columns>
<col>     ::= [A-Za-z]+
<table>   ::= [A-Za-z]+
<cond>    ::= <col> = <value>
<value>   ::= [0-9]+ | '[A-Za-z]+'
```

SELECT | <columns> | FROM | <table> | WHERE | <cond>

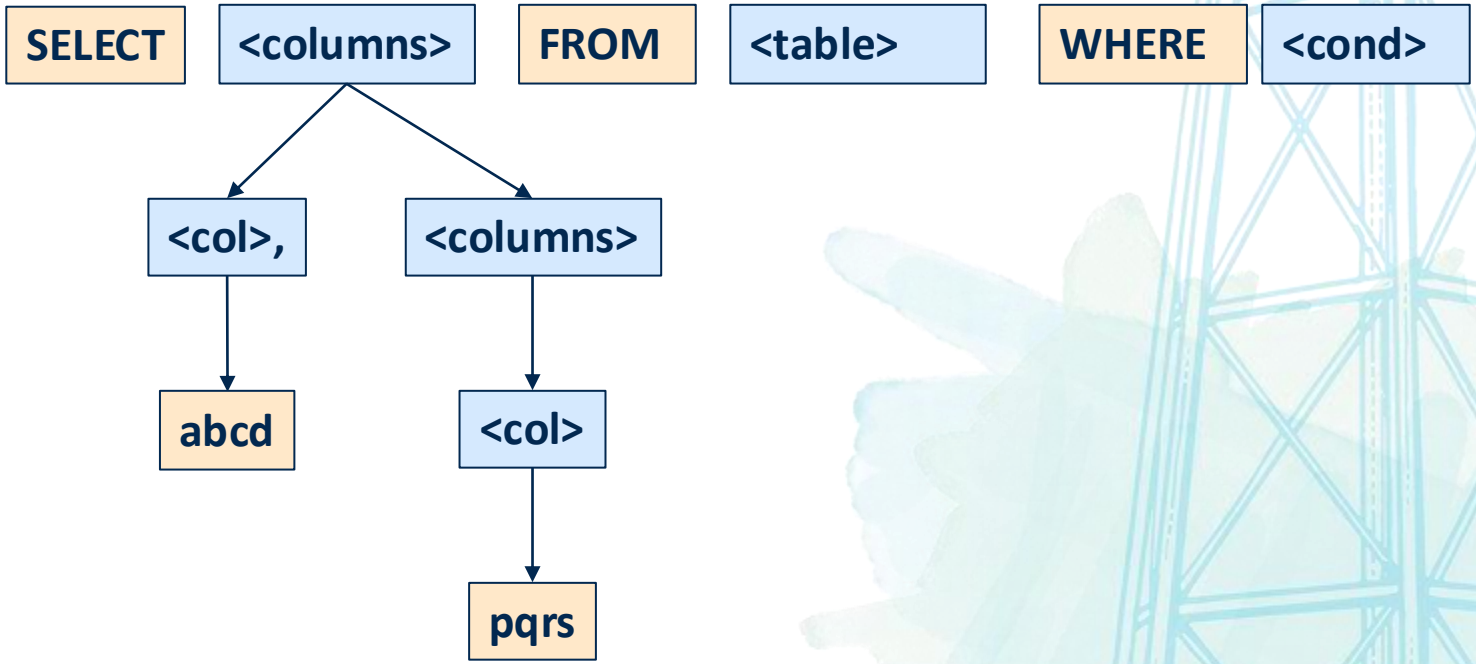
Context free grammar (CFG)

```
<query>   ::= SELECT <columns> FROM <table>
            | SELECT <columns> FROM <table>
              WHERE <cond>
<columns> ::= <col> | <col>, <columns>
<col>     ::= [A-Za-z]+
<table>   ::= [A-Za-z]+
<cond>    ::= <col> = <value>
<value>   ::= [0-9]+ | '[A-Za-z]+'
```



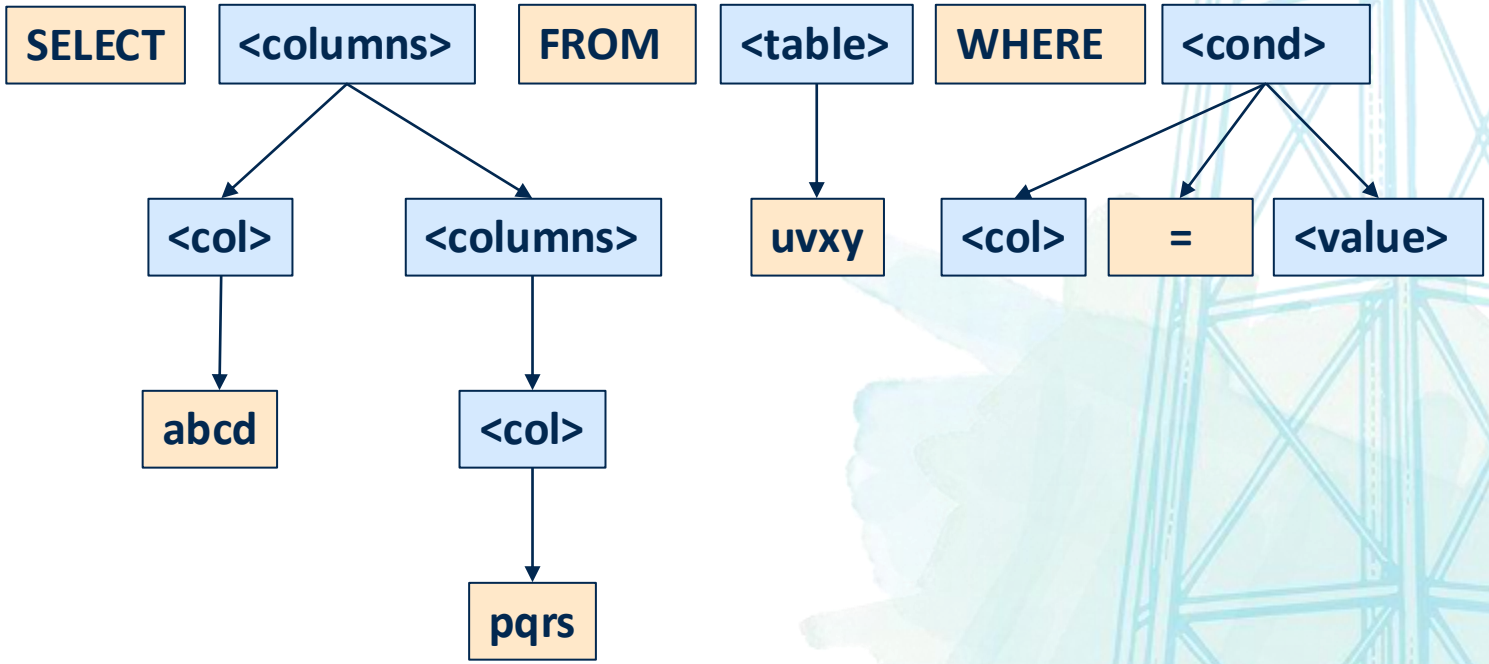
Context free grammar (CFG)

```
<query>   ::= SELECT <columns> FROM <table>
            | SELECT <columns> FROM <table>
              WHERE <cond>
<columns> ::= <col> | <col>, <columns>
<col>     ::= [A-Za-z]+
<table>   ::= [A-Za-z]+
<cond>    ::= <col> = <value>
<value>   ::= [0-9]+ | '[A-Za-z]+'
```



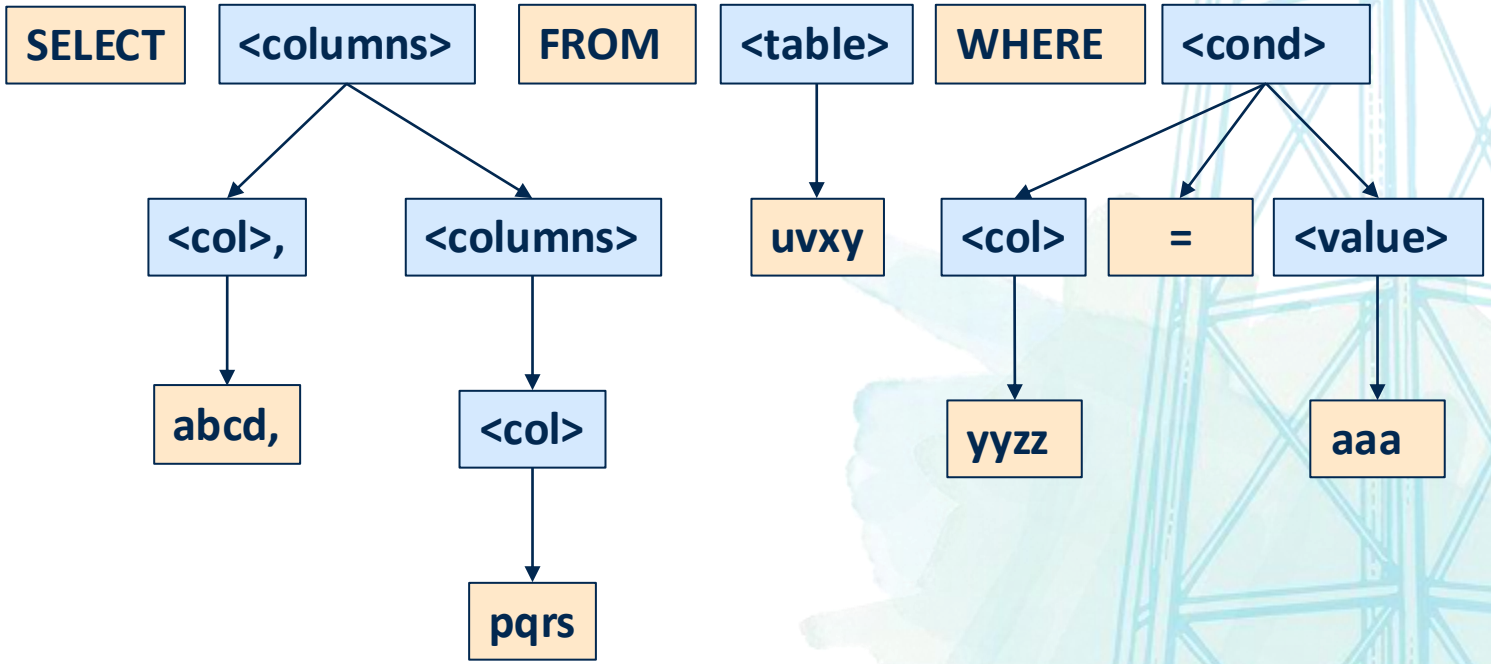
Context free grammar (CFG)

```
<query>   ::= SELECT <columns> FROM <table>
            | SELECT <columns> FROM <table>
               WHERE <cond>
<columns> ::= <col> | <col>, <columns>
<col>     ::= [A-Za-z]+
<table>   ::= [A-Za-z]+
<cond>    ::= <col> = <value>
<value>   ::= [0-9]+ | '[A-Za-z]+'
```



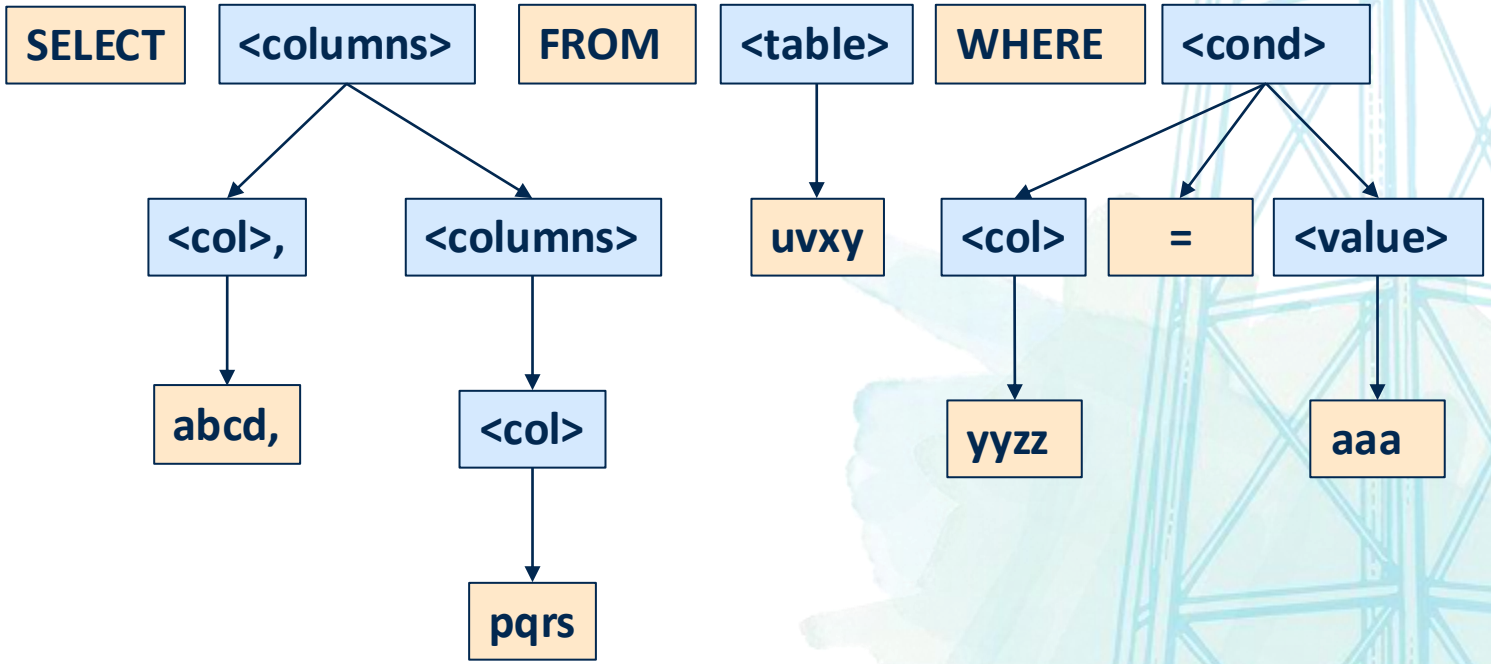
Context free grammar (CFG)

```
<query>   ::= SELECT <columns> FROM <table>
            | SELECT <columns> FROM <table>
              WHERE <cond>
<columns> ::= <col> | <col>, <columns>
<col>     ::= [A-Za-z]+
<table>   ::= [A-Za-z]+
<cond>    ::= <col> = <value>
<value>   ::= [0-9]+ | '[A-Za-z]+'
```



Context free grammar (CFG)

```
<query>   ::= SELECT <columns> FROM <table>
            | SELECT <columns> FROM <table>
              WHERE <cond>
<columns> ::= <col> | <col>, <columns>
<col>     ::= [A-Za-z]+
<table>   ::= [A-Za-z]+
<cond>    ::= <col> = <value>
<value>   ::= [0-9]+ | '[A-Za-z]+'
```



Produced SQL statement: SELECT abcd, pqrs FROM uvxy WHERE yyzz = aaa

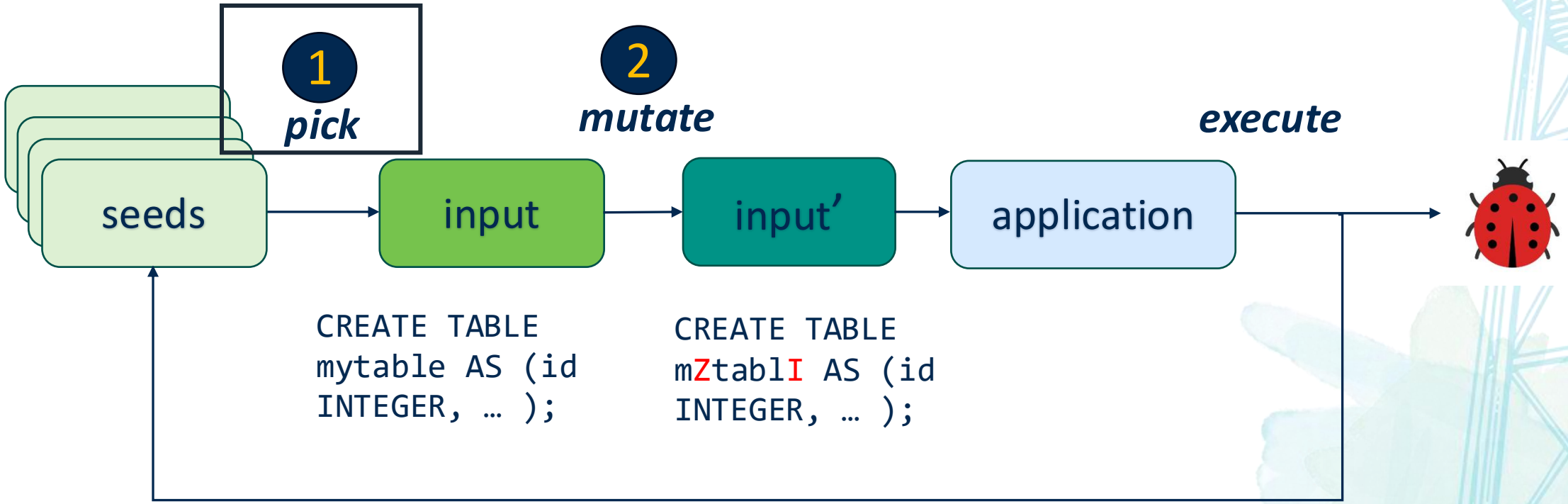
Pros and cons

- Positives:
  - 100% syntactically valid → tests deeper code paths
  - Can weight rules to target specific features
- Negatives
  - Must write/maintain a grammar for each input format
  - Only generates valid inputs — may miss bugs triggered by malformed input
  - Generating and mutating valid inputs is expensive

Fuzzer success factors

- Fuzzer throughput (exec/s)
- Percentage of valid inputs generated by the mutator
- Tradeoff: simpler mutators have higher throughput, but lower number of valid inputs

Fuzzing pipeline



How to pick which seeds to mutate?

- Goal: Should pick seeds that are “most promising” and likely to find bugs
- What is ***“most promising”?***

Metric of seed quality: code coverage

- Code coverage is the simplest metric for measuring seed quality
- Amount of code or execution paths explored during fuzz testing
- Input seeds that increase coverage are “more promising”
  - Explore new program behaviors
  - Ensures untested paths are tested
- High coverage = high bug-finding potential

Metric of seed quality: code coverage

- Amount of code or execution paths explored during fuzz testing

```
int function(char* s) {
    if (s[0] == ‘A’) {
        printf(“Hey!”);
    } else {
        if (s[1] == ‘B’) {
            printf(“Hi!”);
        } else if (s[1] == ‘C’) {
            printf(“Hello!”);
        }
    }
}
```

Metric of seed quality: code coverage

- Amount of code or execution paths explored during fuzz testing
- Input: s = “AB”
  - New coverage! -> “interesting” seed!

```
int function(char* s) {
    if (s[0] == ‘A’) {
        printf(“Hey!”);
    } else {
        if (s[1] == ‘B’) {
            printf(“Hi!”);
        } else if (s[1] == ‘C’) {
            printf(“Hello!”);
        }
    }
}
```

Metric of seed quality: code coverage

- Amount of code or execution paths explored during fuzz testing
- Input: s = “AC”
  - No new coverage -> uninteresting seed!

```
int function(char* s) {
    if (s[0] == ‘A’) {
        printf(“Hey!”);
    } else {
        if (s[1] == ‘B’) {
            printf(“Hi!”);
        } else if (s[1] == ‘C’) {
            printf(“Hello!”);
        }
    }
}
```

Metric of seed quality: code coverage

- Amount of code or execution paths explored during fuzz testing
- Input: s = “BB”
  - New coverage! -> “interesting” seed!

```
int function(char* s) {
    if (s[0] == ‘A’) {
        printf(“Hey!”);
    } else {
        if (s[1] == ‘B’) {
            printf(“Hi!”);
        } else if (s[1] == ‘C’) {
            printf(“Hello!”);
        }
    }
}
```

Metric of seed quality: code coverage

- Amount of code or execution paths explored during fuzz testing
- Input: s = “BA”
  - No new coverage -> uninteresting seed!

```
int function(char* s) {
    if (s[0] == ‘A’) {
        printf(“Hey!”);
    } else {
        if (s[1] == ‘B’) {
            printf(“Hi!”);
        } else if (s[1] == ‘C’) {
            printf(“Hello!”);
        }
    }
}
```

Metric of seed quality: code coverage

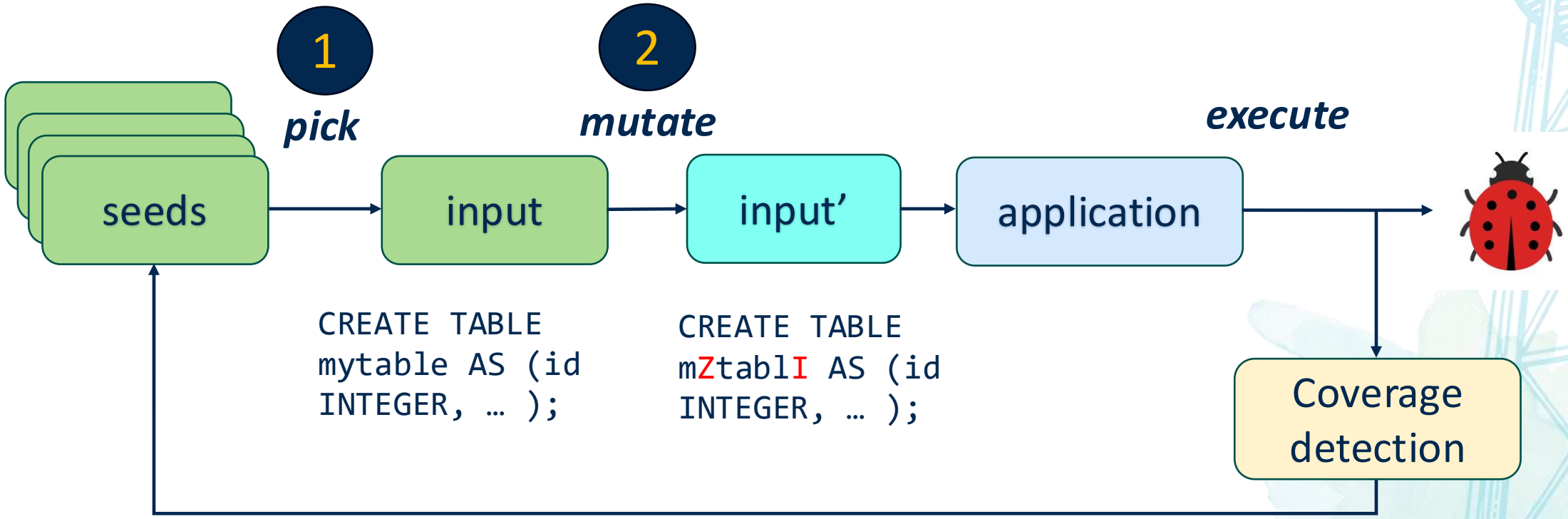
- Amount of code or execution paths explored during fuzz testing
- Input: s = “BC”
  - New coverage! -> “interesting” seed!

```
int function(char* s) {
    if (s[0] == ‘A’) {
        printf(“Hey!”);
    } else {
        if (s[1] == ‘B’) {
            printf(“Hi!”);
        } else if (s[1] == ‘C’) {
            printf(“Hello!”);
        }
    }
}
```

Coverage guided fuzzing

- Record the coverage
- Prioritize mutating input seeds which give higher coverage
- Gradually uncover all program paths
- Also known as “greybox fuzzing”

Coverage-guided fuzzer pipeline



Add* input' *to* seeds *if new coverage observed

Fuzzing – high level ideas

- Randomly generate inputs
- Mutation fuzzing
  - Start with known valid inputs
  - Mutate them
    - Byte-level transformations
    - Dictionary-based transformations
- Pick inputs which resulted in new branch coverage for further mutation

AFL demo

- American fuzzy lop (AFL)
- Fuzzer for C/C++ applications
- https://github.com/wolframroesler/afl-demo

AFL demo

- Throughput – number of inputs tested per second

```
                    american fuzzy lop 2.52b (afldemo)

┌─ process timing ─────────────────────────────────────┬─ overall results ─────┐
│        run time : 0 days, 0 hrs, 0 min, 46 sec       │  cycles done : 0      │
│   last new path : 0 days, 0 hrs, 0 min, 0 sec        │  total paths : 309    │
│ last uniq crash : 0 days, 0 hrs, 0 min, 0 sec        │ uniq crashes : 215    │
│  last uniq hang : none seen yet                      │   uniq hangs : 0      │
├─ cycle progress ────────────────────┬─ map coverage ─┴───────────────────────┤
│  now processing : 27 (8.74%)        │    map density : 0.06% / 2.23%         │
│ paths timed out : 0 (0.00%)         │ count coverage : 1.08 bits/tuple       │
├─ stage progress ────────────────────┼─ findings in depth ────────────────────┤
│  now trying : havoc                 │ favored paths : 107 (34.63%)           │
│ stage execs : 12.9k/32.8k (39.37%)  │  new edges on : 287 (92.88%)           │
│ total execs : 128k                  │ total crashes : 2882 (215 unique)      │
│  exec speed : 2142/sec              │  total tmouts : 0 (0 unique)           │
├─ fuzzing strategy yields ───────────┴───────────────┬─ path geometry ────────┤
│   bit flips : 24/2024, 22/2016, 20/2000             │    levels : 4          │
│  byte flips : 3/253, 1/245, 0/231                   │   pending : 302        │
│ arithmetics : 167/14.2k, 0/1532, 0/140              │  pend fav : 101        │
│  known ints : 17/1368, 6/6782, 2/10.1k              │ own finds : 306        │
│  dictionary : 0/0, 0/0, 0/0                         │  imported : n/a        │
│       havoc : 193/71.9k, 0/0                        │ stability : 100.00%    │
│        trim : 0.79%/86, 0.00%                       ├────────────────────────┘
└─────────────────────────────────────────────────────┘          [cpu000: 31%]
```

AFL demo

- Crashes found by the fuzzer

```
                    american fuzzy lop 2.52b (afldemo)

┌─ process timing ─────────────────────────────────────┬─ overall results ─────┐
│        run time : 0 days, 0 hrs, 0 min, 46 sec       │  cycles done : 0      │
│   last new path : 0 days, 0 hrs, 0 min, 0 sec        │  total paths : 309    │
│ last uniq crash : 0 days, 0 hrs, 0 min, 0 sec        │ uniq crashes : 215    │
│  last uniq hang : none seen yet                      │   uniq hangs : 0      │
├─ cycle progress ────────────────────┬─ map coverage ─┴───────────────────────┤
│  now processing : 27 (8.74%)        │    map density : 0.06% / 2.23%         │
│ paths timed out : 0 (0.00%)         │ count coverage : 1.08 bits/tuple       │
├─ stage progress ────────────────────┼─ findings in depth ────────────────────┤
│  now trying : havoc                 │ favored paths : 107 (34.63%)           │
│ stage execs : 12.9k/32.8k (39.37%)  │  new edges on : 287 (92.88%)           │
│ total execs : 128k                  │ total crashes : 2882 (215 unique)      │
│  exec speed : 2142/sec              │  total tmouts : 0 (0 unique)           │
├─ fuzzing strategy yields ───────────┴───────────────┬─ path geometry ────────┤
│   bit flips : 24/2024, 22/2016, 20/2000             │    levels : 4          │
│  byte flips : 3/253, 1/245, 0/231                   │   pending : 302        │
│ arithmetics : 167/14.2k, 0/1532, 0/140              │  pend fav : 101        │
│  known ints : 17/1368, 6/6782, 2/10.1k              │ own finds : 306        │
│  dictionary : 0/0, 0/0, 0/0                         │  imported : n/a        │
│       havoc : 193/71.9k, 0/0                        │ stability : 100.00%    │
│        trim : 0.79%/86, 0.00%                       ├────────────────────────┘
└─────────────────────────────────────────────────────┘          [cpu000: 31%]
```

Crashes

AFL demo

- Coverage – number of code paths executed

```
                    american fuzzy lop 2.52b (afldemo)

┌─ process timing ─────────────────────────────────────┬─ overall results ─────┐
│        run time : 0 days, 0 hrs, 0 min, 46 sec       │  cycles done : 0      │
│   last new path : 0 days, 0 hrs, 0 min, 0 sec        │  total paths : 309    │
│ last uniq crash : 0 days, 0 hrs, 0 min, 0 sec        │ uniq crashes : 215    │
│  last uniq hang : none seen yet                      │   uniq hangs : 0      │
├─ cycle progress ────────────────────┬─ map coverage ─┴───────────────────────┤
│  now processing : 27 (8.74%)        │    map density : 0.06% / 2.23%         │
│ paths timed out : 0 (0.00%)         │ count coverage : 1.08 bits/tuple       │
├─ stage progress ────────────────────┼─ findings in depth ────────────────────┤
│  now trying : havoc                 │ favored paths : 107 (34.63%)           │
│ stage execs : 12.9k/32.8k (39.37%)  │  new edges on : 287 (92.88%)           │
│ total execs : 128k                  │ total crashes : 2882 (215 unique)      │
│  exec speed : 2142/sec              │  total tmouts : 0 (0 unique)           │
├─ fuzzing strategy yields ───────────┴───────────────┬─ path geometry ────────┤
│   bit flips : 24/2024, 22/2016, 20/2000             │    levels : 4          │
│  byte flips : 3/253, 1/245, 0/231                   │   pending : 302        │
│ arithmetics : 167/14.2k, 0/1532, 0/140              │  pend fav : 101        │
│  known ints : 17/1368, 6/6782, 2/10.1k              │ own finds : 306        │
│  dictionary : 0/0, 0/0, 0/0                         │  imported : n/a        │
│       havoc : 193/71.9k, 0/0                        │ stability : 100.00%    │
│        trim : 0.79%/86, 0.00%                       ├────────────────────────┘
└─────────────────────────────────────────────────────┘          [cpu000: 31%]
```

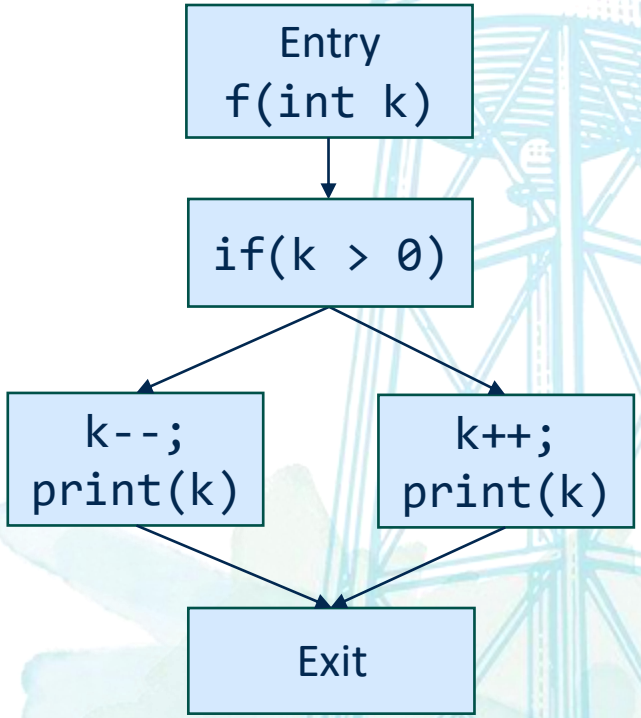
Coverage

Coverage collection

Control flow graph

- A program can have many representations
- Control flow graph is the one of the simplest of representations
- A CFG consists of basic blocks

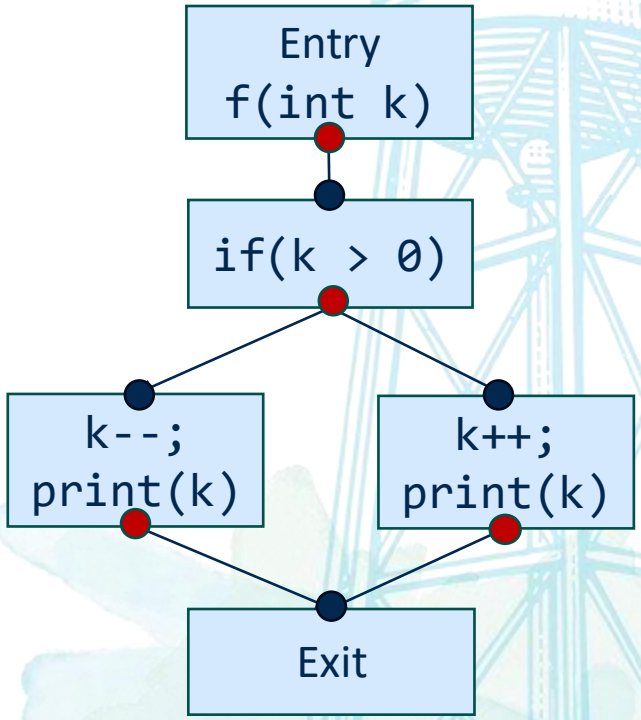
```
void f(int k) {
    if (k > 0) {
        k--;
        print(k);
    } else {
        k++;
        print(k);
    }
}
```



Basic blocks

- A basic block in program P is a sequence of consecutive statements
- A BB has a single entry and a single exit point

```
void f(int k) {
    if (k > 0) {
        k--;
        print(k);
    } else {
        k++;
        print(k);
    }
}
```

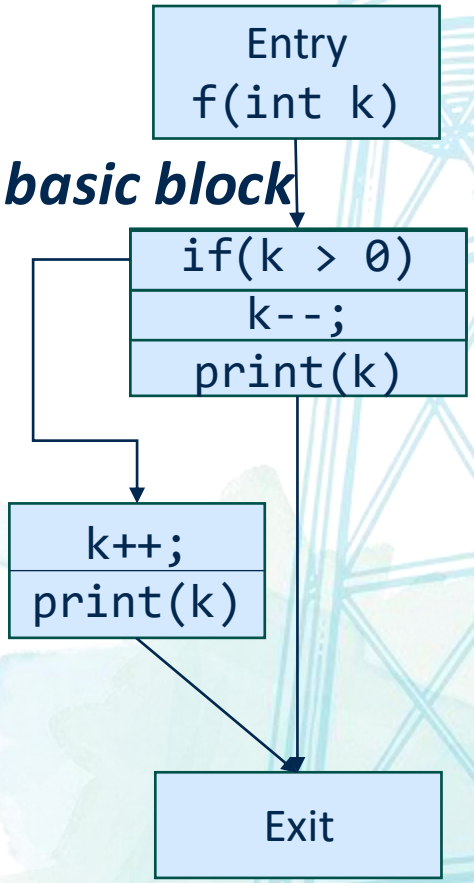


Basic blocks

- A basic block in program P is a sequence of consecutive statements
- A BB has a single entry and a single exit point

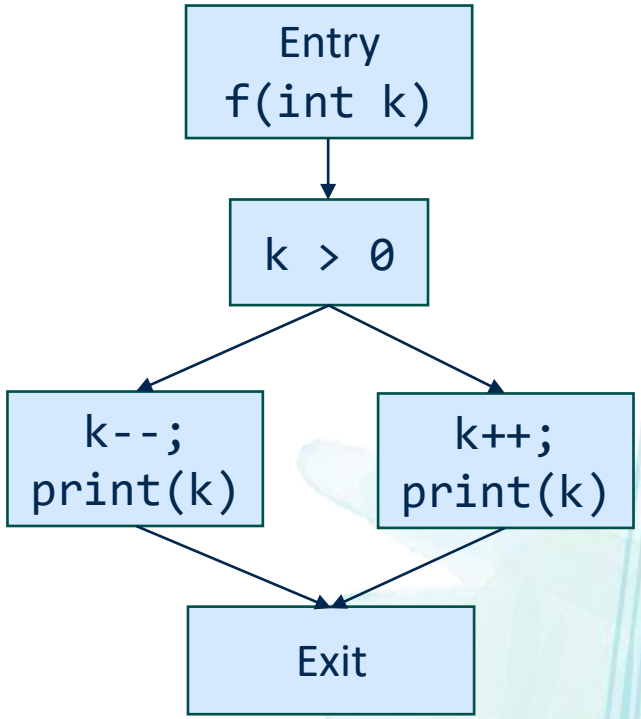
```
void f(int k) {
    if (k > 0) {
        k--;
        print(k);
    } else {
        k++;
        print(k);
    }
}
```

Not a basic block



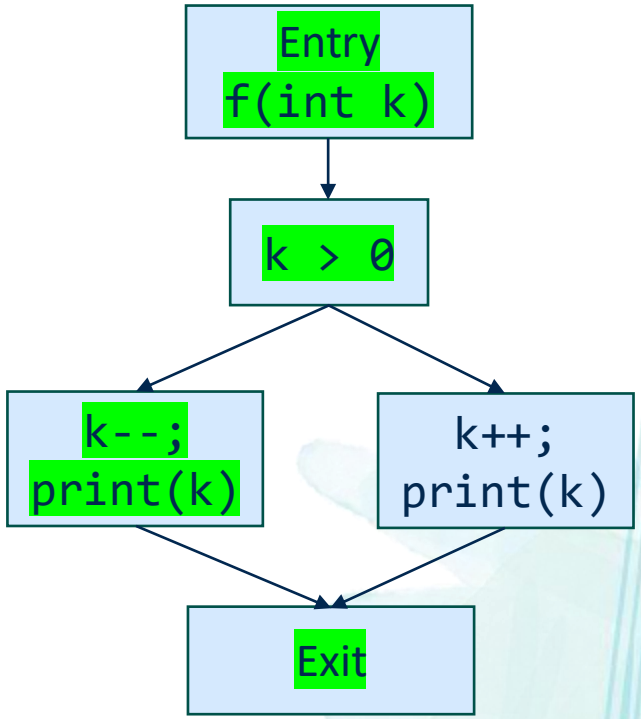
BB-level coverage

- Fuzzers record coverage at the BB level
- For every input, the fuzzer records which BBs are covered
- If new BBs are covered, it considers the input “interesting”
- E.g. `f(10)`



BB-level coverage

- Fuzzers typically record coverage at the BB level
- For every input, the fuzzer records which BBs are covered
- If new BBs are covered, it considers the input “interesting”
- E.g. `f(10)`

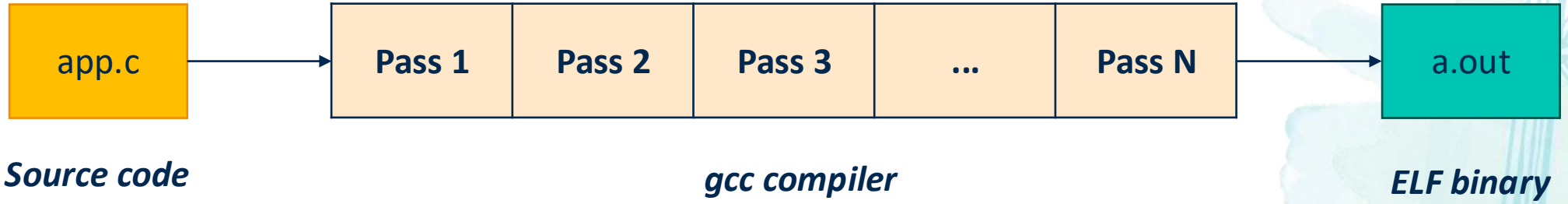


Compilers

Many automated software engineering techniques, including fuzzing, leverage the compiler

Compilers

- Compiler consists of “passes” that analyze and transform the code
- Each pass gradually lowers the source code to assembly
  - Produces many program representations, including CFG



Coverage instrumentation

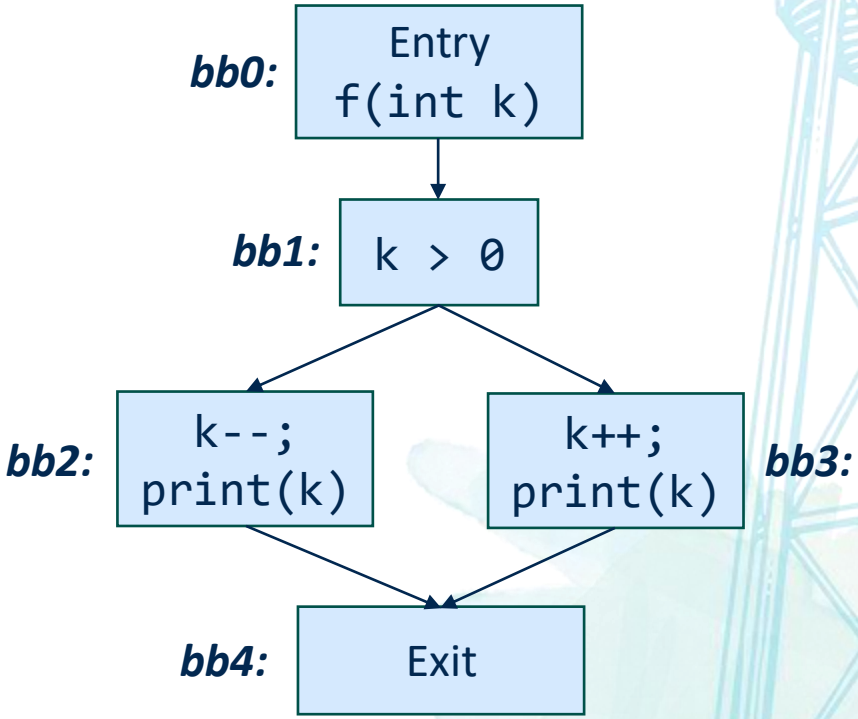
- `afl-gcc` is a wrapper around `gcc`
- Inserts additional passes into the compiler pipeline

Coverage instrumentation

- AFL compiler instruments the CFG edge-transition to record coverage
- Adds a coverage map

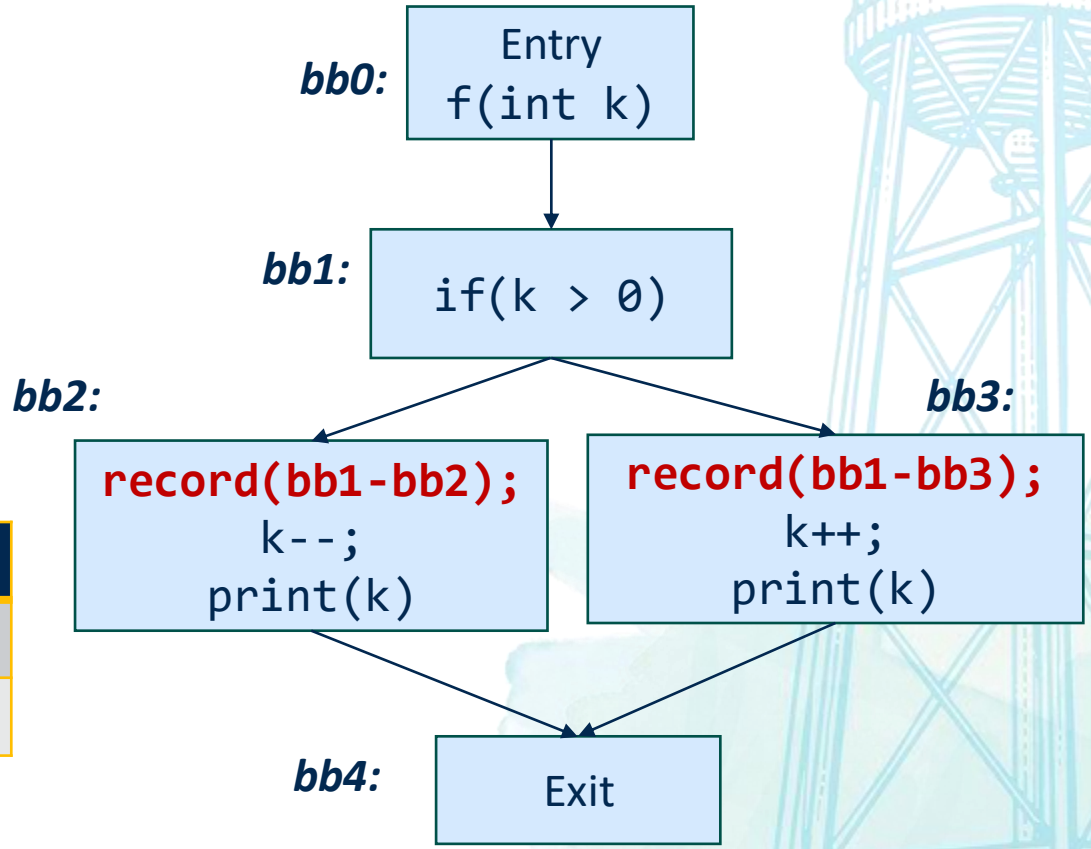
Edge	Coverage
bb1-bb2	
bb1-bb3	

Coverage map



Coverage instrumentation

- AFL compiler instruments the CFG edge-transition to record coverage
- Adds a coverage map



Edge	Coverage
bb1-bb2	
bb1-bb3	

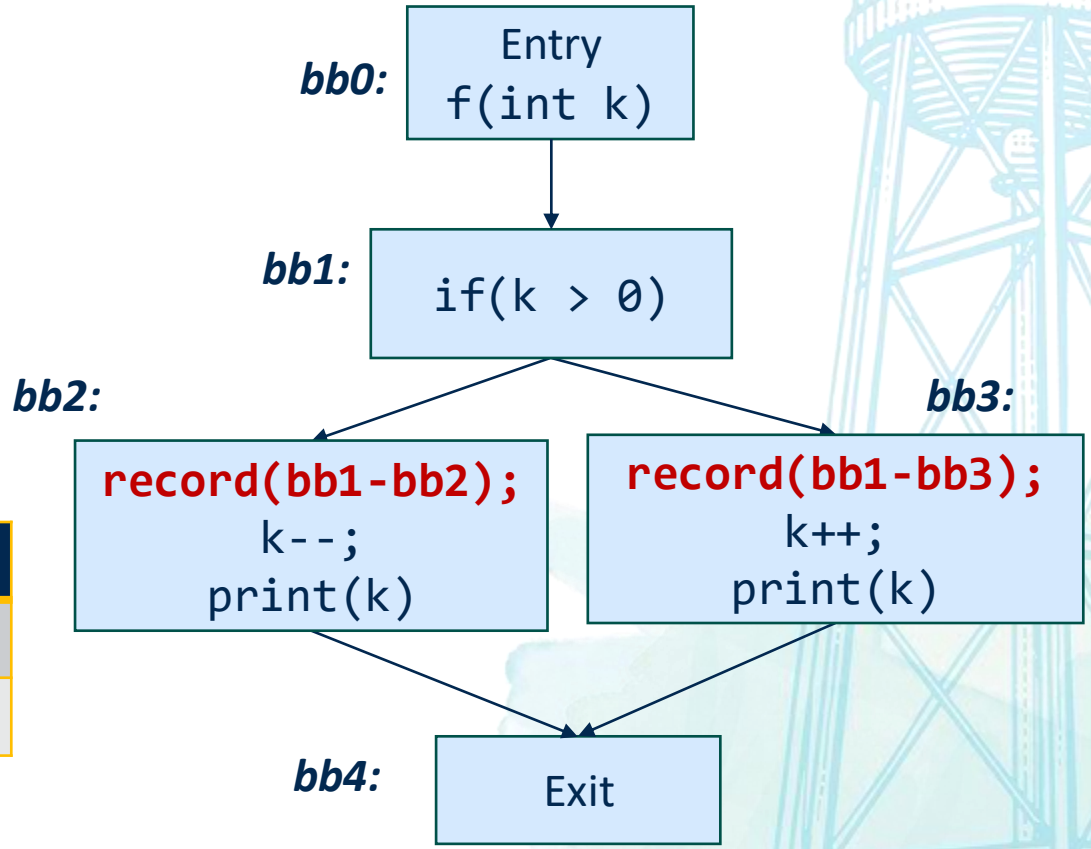
Coverage map

Coverage recording during execution

- Instrumentation records the coverage in the coverage map
- Fuzzer reads the coverage map to determine which edges covered
- E.g. `f(10)`

Edge	Coverage
bb1-bb2	
bb1-bb3	

Coverage map

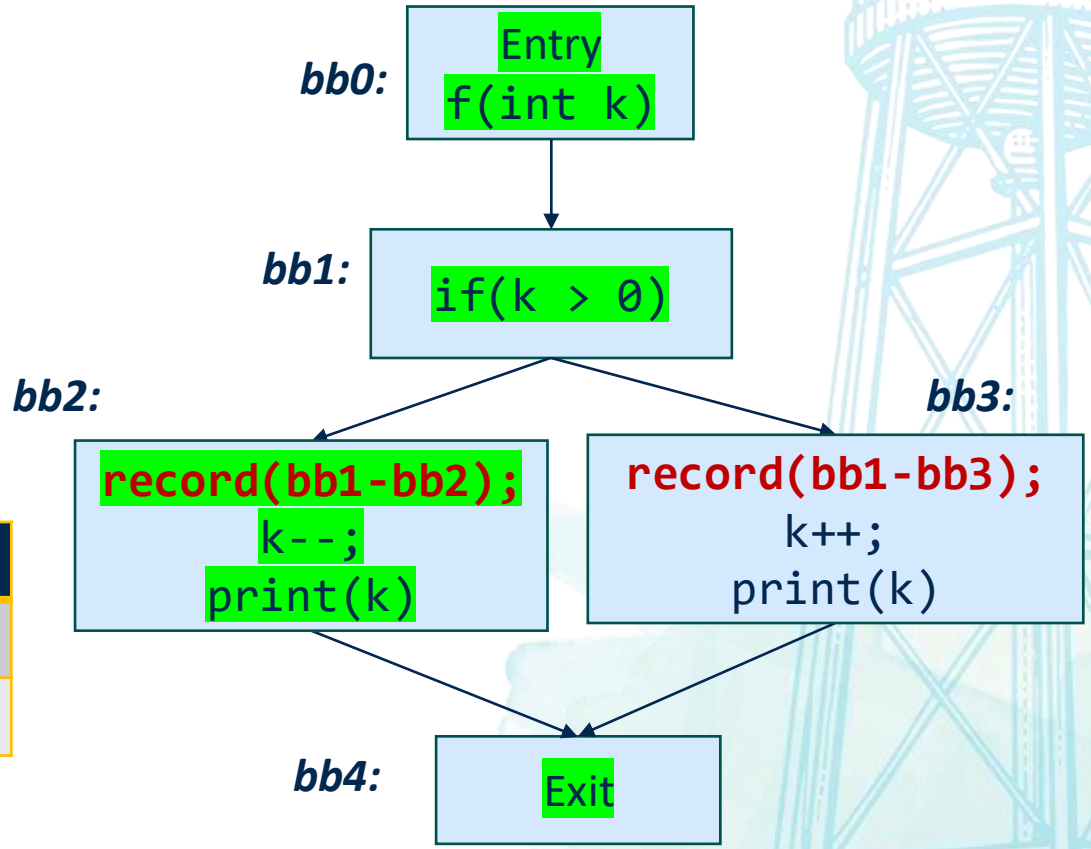


Coverage recording during execution

- Instrumentation records the coverage in the coverage map
- Fuzzer reads the coverage map to determine which edges covered
- E.g. `f(10)`

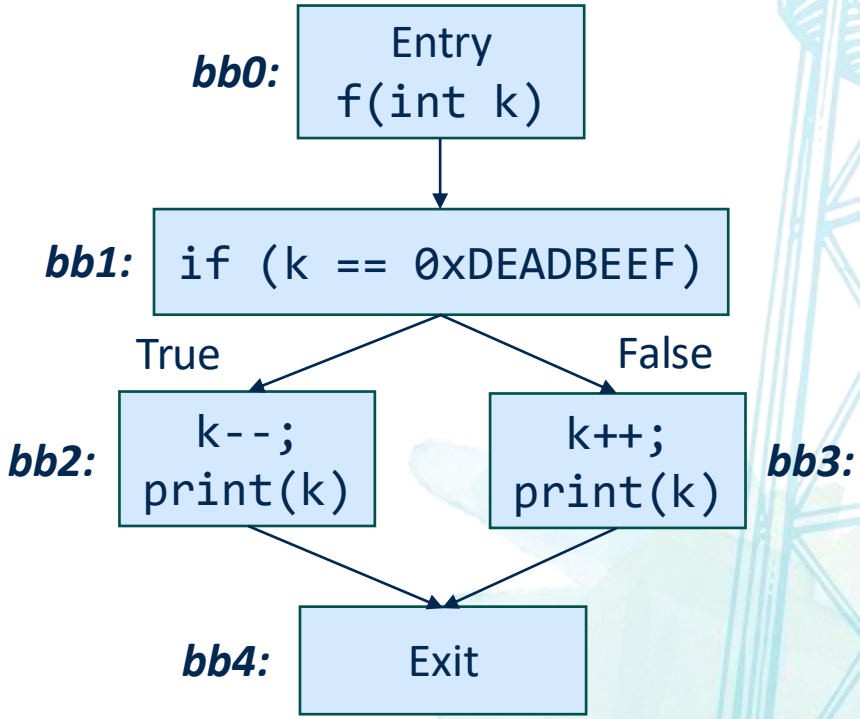
Edge	Coverage
bb1-bb2	1
bb1-bb3	

Coverage map



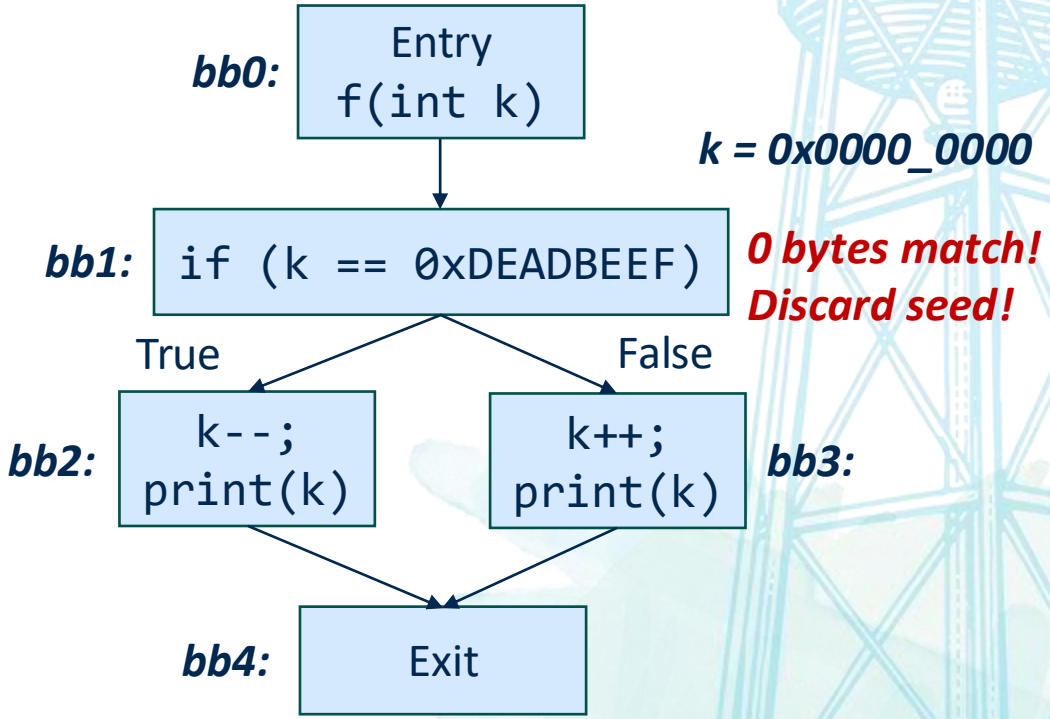
Branch coverage limitation

- E.g., branch condition `if (input == 0xDEADBEEF)`
- Branch coverage sees "not taken" for *every* input except the magic value
- Hard to distinguish 0xDEADBEE0 (close!) from 0x00000000 (far)
- Random mutation is very unlikely to hit the exact value



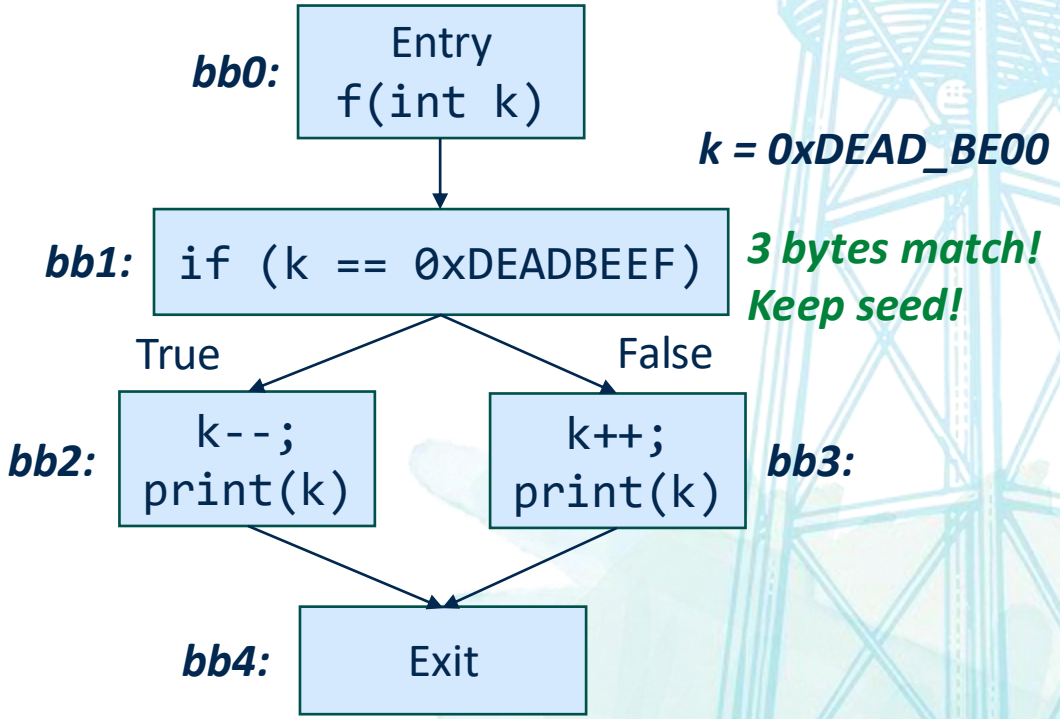
Value profile coverage

- Track the *values* being compared, not just the branch outcome
- For `if (x == 0xDEADBEEF)`, count how many bytes of x match the target
- Used by libfuzzer
- E.g., `f(0x0000_0000)`



Value profile coverage

- Track the *values* being compared, not just the branch outcome
- For `if (x == 0xDEADBEEF)`, count how many bytes of `x` match the target
- E.g., `f(0xDEAD_BE00)`
- Used by libfuzzer
- Value profile gradually "walks" toward the target value



Path coverage

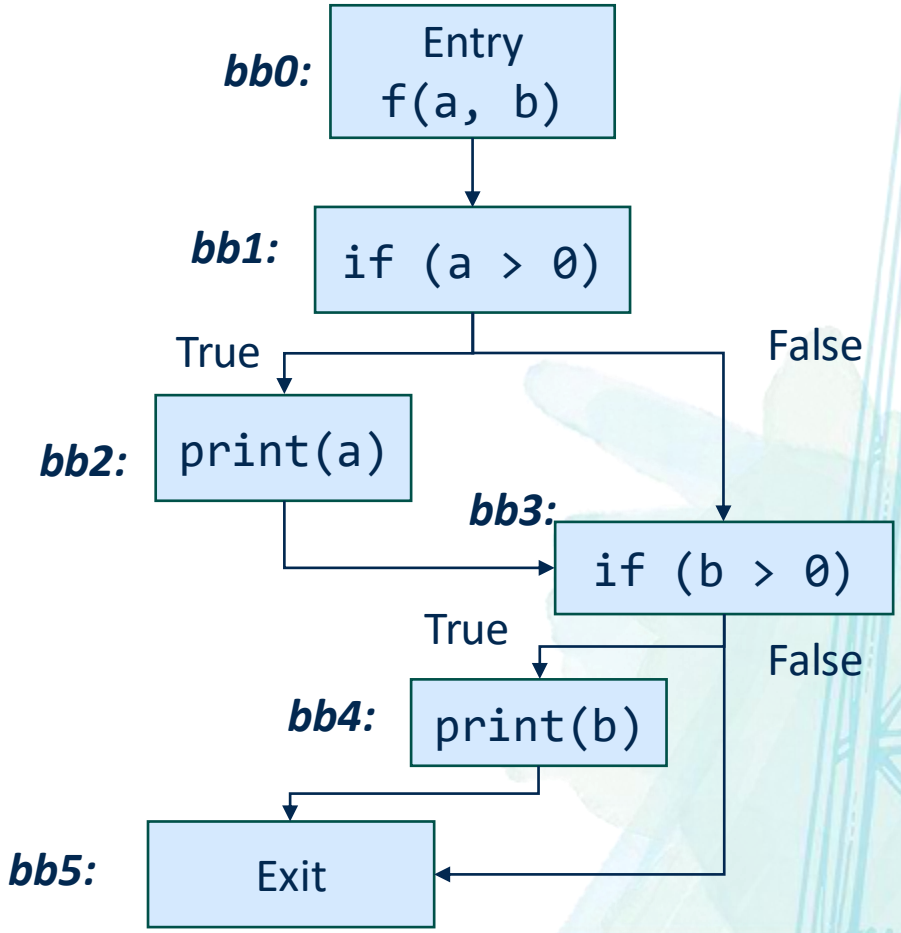
- Branch coverage: did we take this branch (yes/no)?
- Path coverage: track the *exact sequence* of branches taken
- Same branches, different order = different path

```
if (a > 0) { print("a"); }   // branch A
if (b > 0) { print("b"); }   // branch B
```

Path coverage

- Branch coverage: did we take this branch (yes/no)?
- Path coverage: track the *exact sequence* of branches taken
- Same branches, different order = different path
- Bugs might depend on the exact path

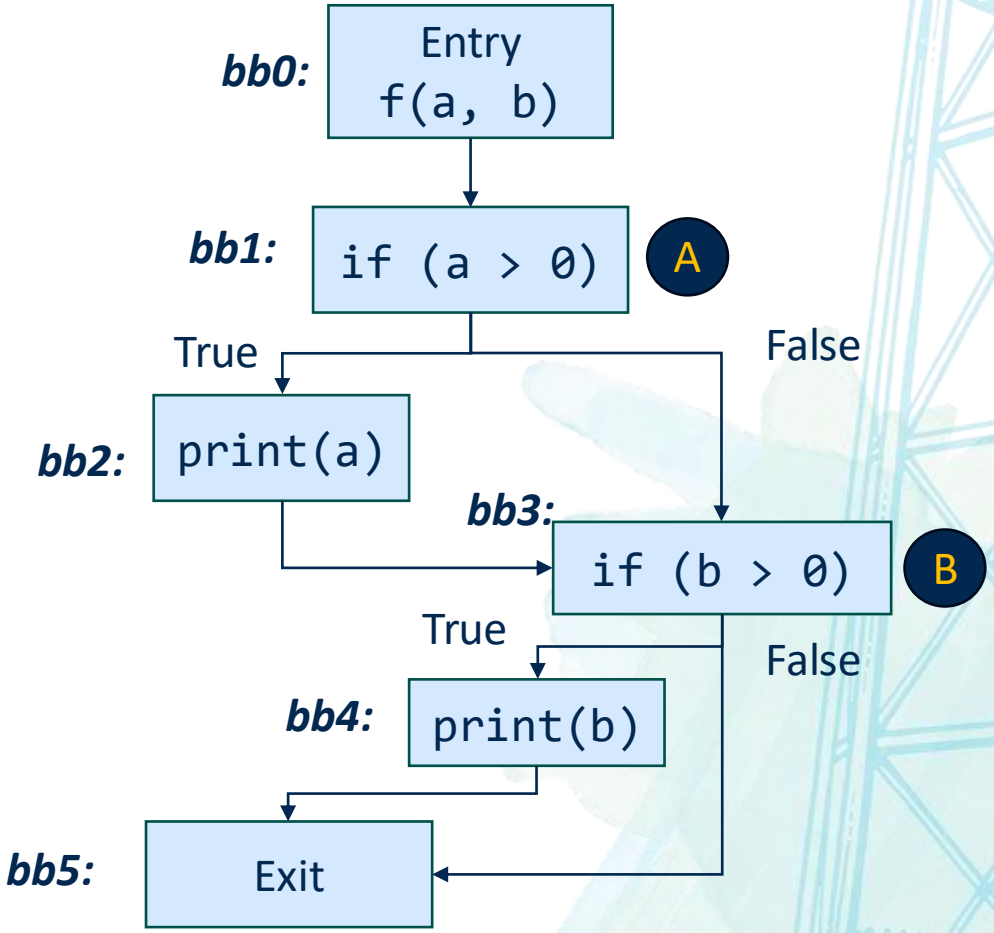
```
if (a > 0) { print("a"); }   // branch A
if (b > 0) { print("b"); }   // branch B
```



Path coverage

- How many paths?
  - A not taken, B not taken
  - A not taken, B taken
  - A taken, B not taken
  - A taken, B taken

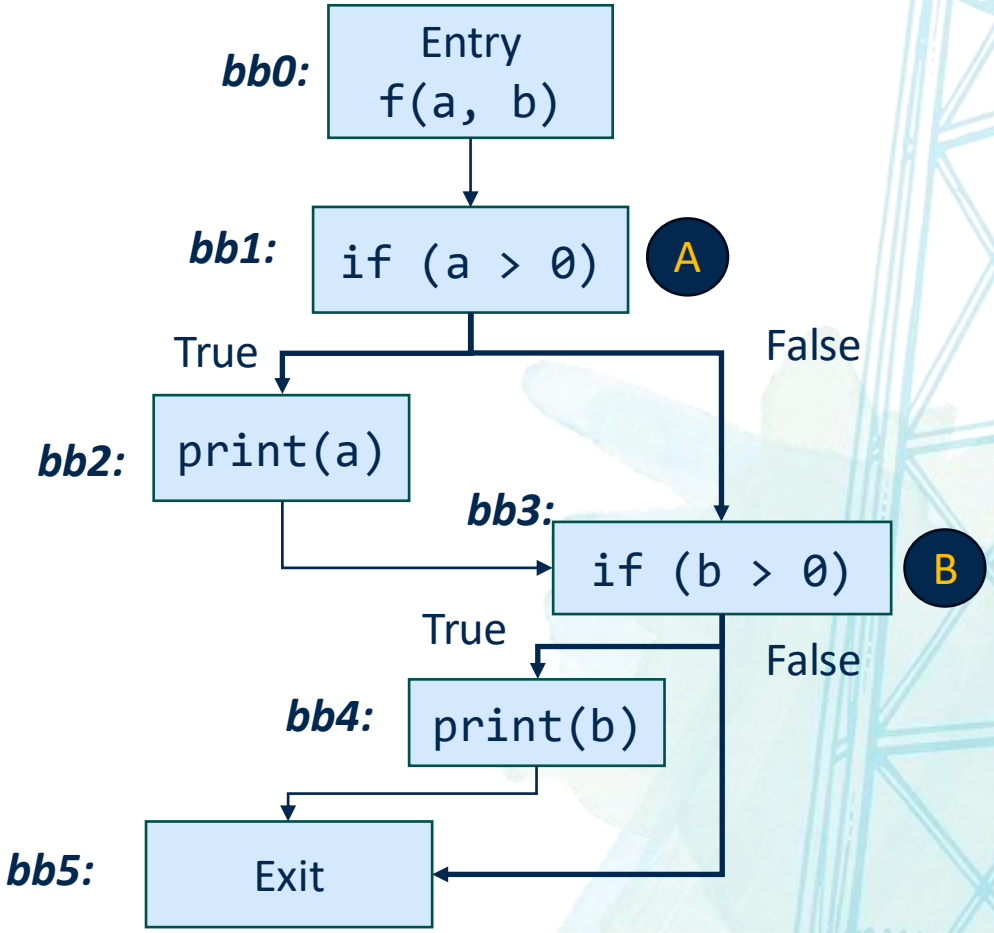
```
if (a > 0) { print("a"); }   // branch A
if (b > 0) { print("b"); }   // branch B
```



Path coverage vs edge coverage

- Edge coverage only records whether an edge was traversed
  - Ignores execution history

```
if (a > 0) { print("a"); }   // branch A
if (b > 0) { print("b"); }   // branch B
```



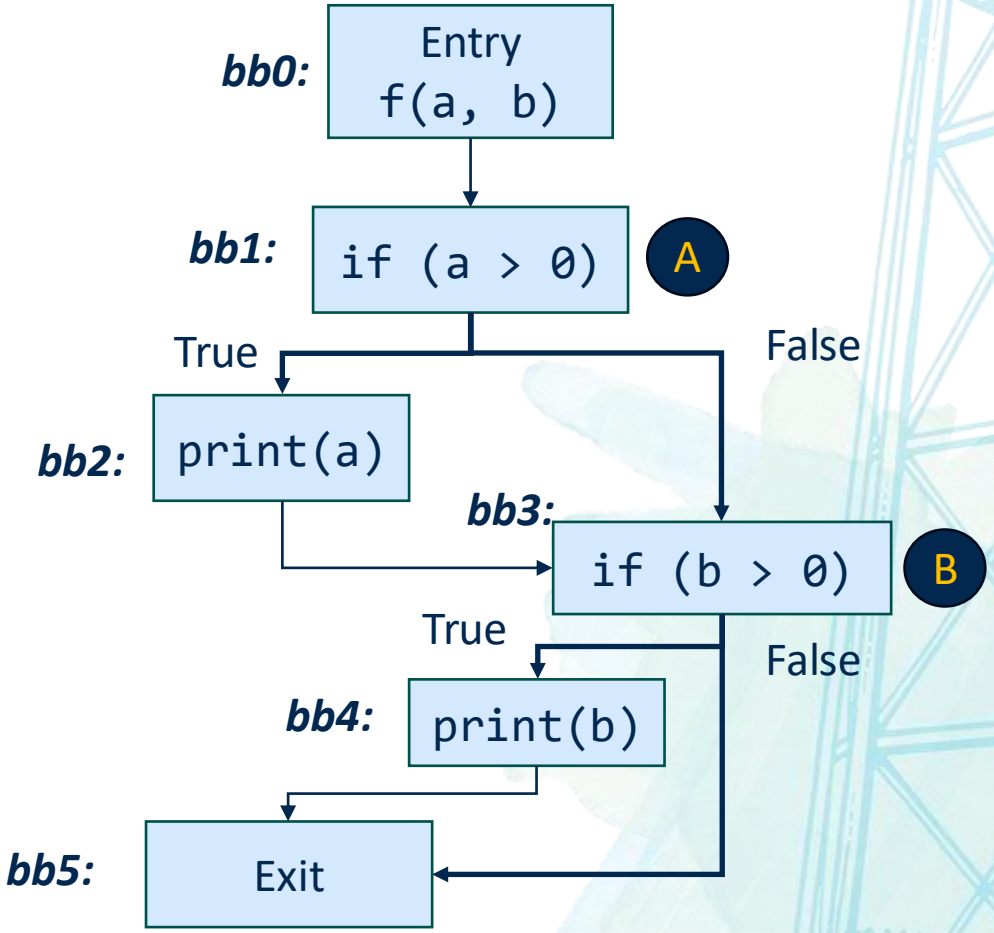
Coverage map

Edge	Coverage
bb1-bb2	
bb1-bb3	
bb3-bb4	
bb3-bb5	

Path coverage vs edge coverage

What is the minimum number of inputs that can result in ”full coverage” for an edge-coverage approach?

```
if (a > 0) { print(“a”); }   // branch A
if (b > 0) { print(“b”); }   // branch B
```



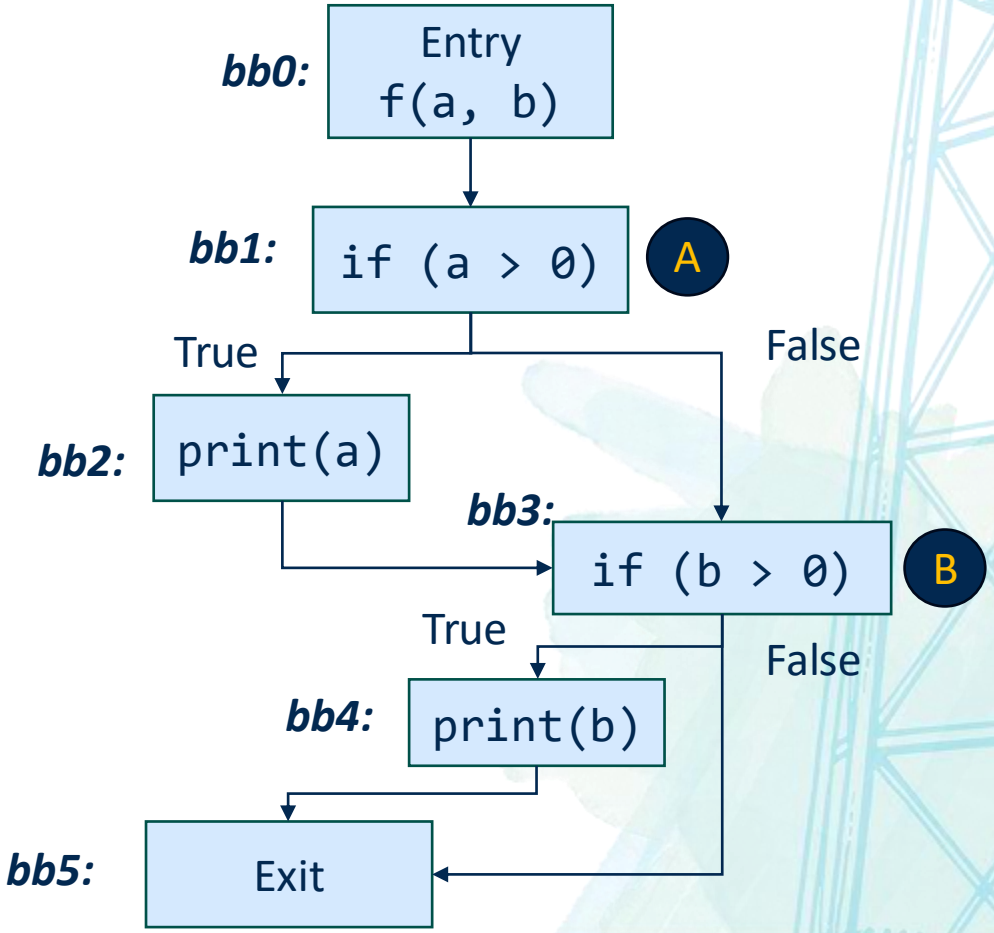
Coverage map

Edge	Coverage
bb1-bb2	
bb1-bb3	
bb3-bb4	
bb3-bb5	

Path coverage vs edge coverage

Eg. a = 10, b = -1

```
if (a > 0) { print("a"); }   // branch A
if (b > 0) { print("b"); }   // branch B
```



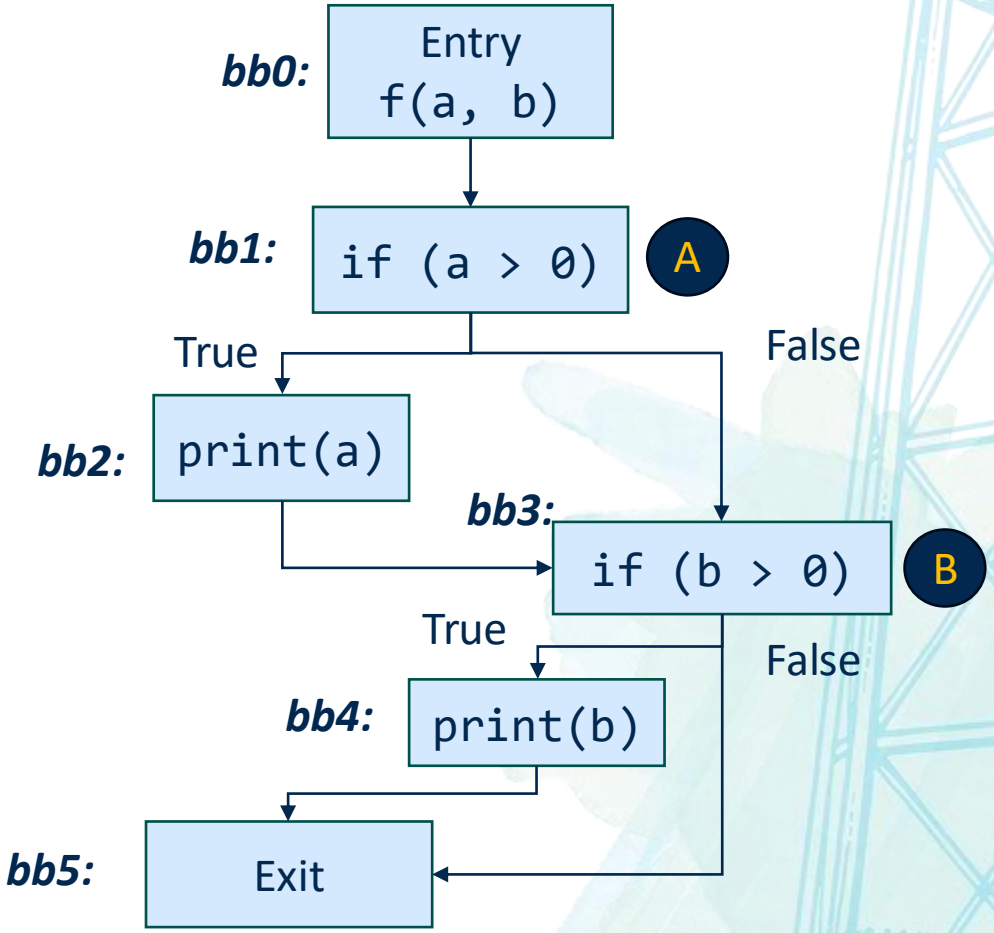
Coverage map

Edge	Coverage
bb1-bb2	1
bb1-bb3	0
bb3-bb4	0
bb3-bb5	1

Path coverage vs edge coverage

Eg. a = -1, b = 10

```
if (a > 0) { print("a"); }   // branch A
if (b > 0) { print("b"); }   // branch B
```



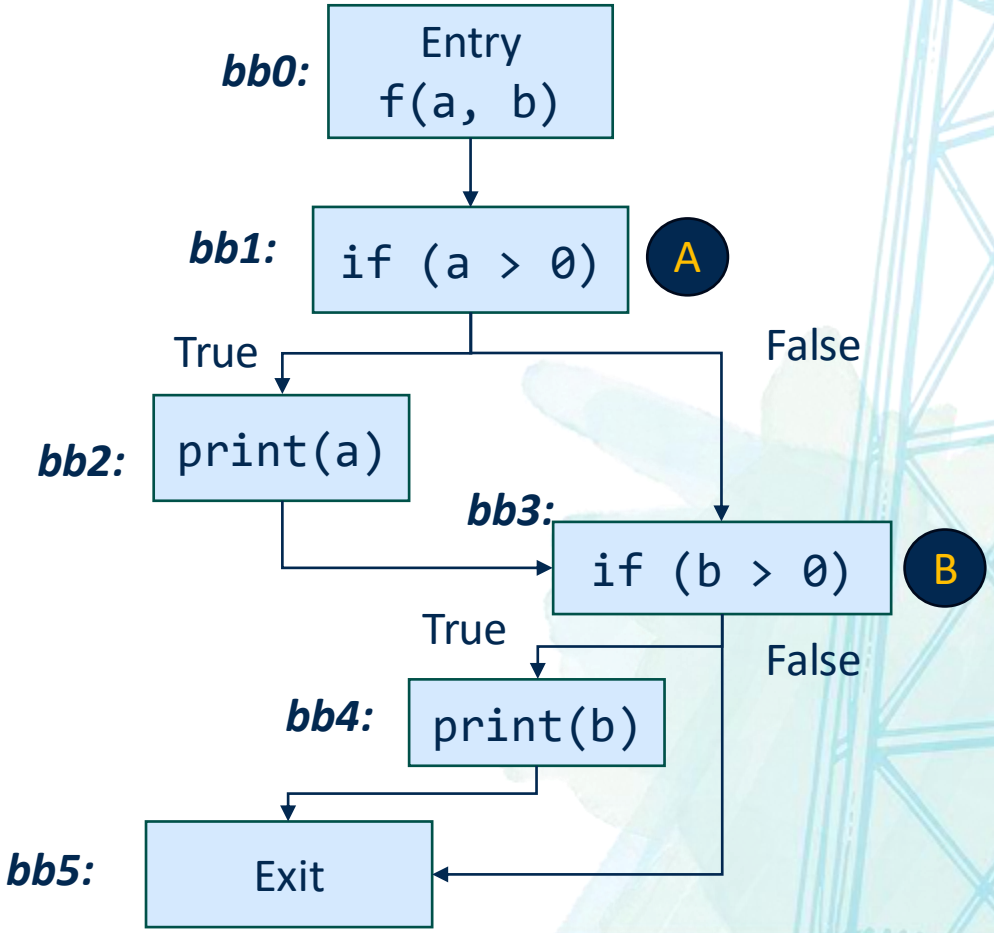
Coverage map

Edge	Coverage
bb1-bb2	1
bb1-bb3	1
bb3-bb4	1
bb3-bb5	1

Path coverage vs edge coverage

- Two inputs result in "full" coverage
- But explores only 2 of 4 paths

```
if (a > 0) { print("a"); }   // branch A
if (b > 0) { print("b"); }   // branch B
```



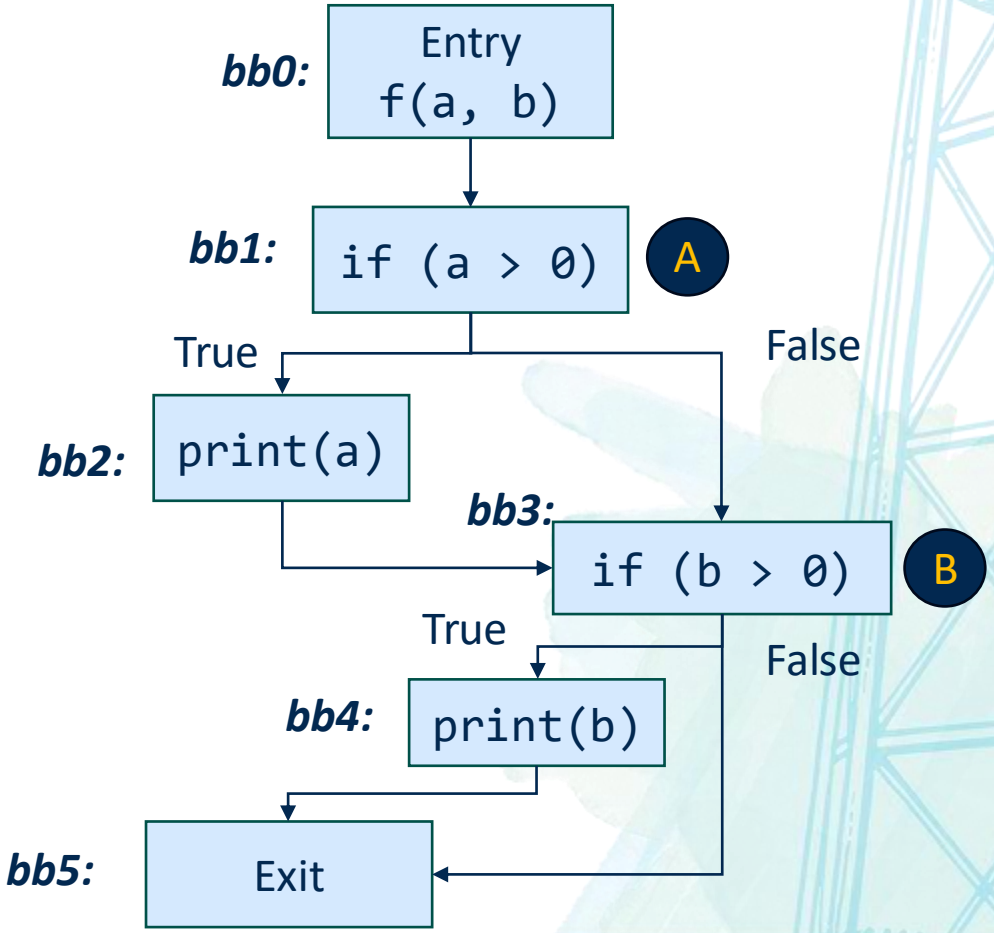
Coverage map

Edge	Coverage
bb1-bb2	1
bb1-bb3	1
bb3-bb4	1
bb3-bb5	1

Path coverage vs edge coverage

Path	Explored?
A not taken, B not taken	No
A not taken, B taken	Yes
A taken, B not taken	Yes
A taken, B taken	No

```
if (a > 0) { print("a"); }    // branch A
if (b > 0) { print("b"); }    // branch B
```



Coverage map

Edge	Coverage
bb1-bb2	1
bb1-bb3	1
bb3-bb4	1
bb3-bb5	1

Path coverage

- A path-coverage based fuzzer records path-coverage instead of just edge-coverage
- Stores significantly more information per execution
- Leads to path explosion
  - With N branches, up to 2^N possible paths
  - Loops make matters worse – every iteration is a new path!
- Path coverage is the most precise but impractical to fully achieve
  - Industrial fuzzers like AFL only use edge-coverage for coverage feedback

Coverage granularity spectrum

- Branch/edge coverage: which branches were taken (AFL)
- Value profile: how close comparisons are to being satisfied (libFuzzer)
- Path coverage: exact sequence of branches (most precise, but exponential blowup)
- Fine-grained coverage has a higher performance overhead
- Tradeoffs depend on your application!

Directed greybox fuzzing

Directed greybox fuzzing

- Greybox fuzzing cannot be directed
  - Execution cannot be directed to specific parts of the code
- Directed fuzzing has many applications
  - Patch testing: reach changed statements
  - Crash reproduction: reproduce reported bugs
  - Sensitive functions and APIs

DGF compiler instrumentation

- Extract call graph (CG) and control-flow graph (CFGs)
- For each BB and function, compute distance to target locations
- Instrument program to aggregate distance values

```
void f(int a, int b) {
    if (a > 10) {
        // -- snip
    } else {
        if (b > 100) {
            // -- snip
        } else {
            if (b < 200) {
                bug();
            }
        }
    }
}
void g() { f(); }
void h() { print("hello!"); }
// ... snip
```

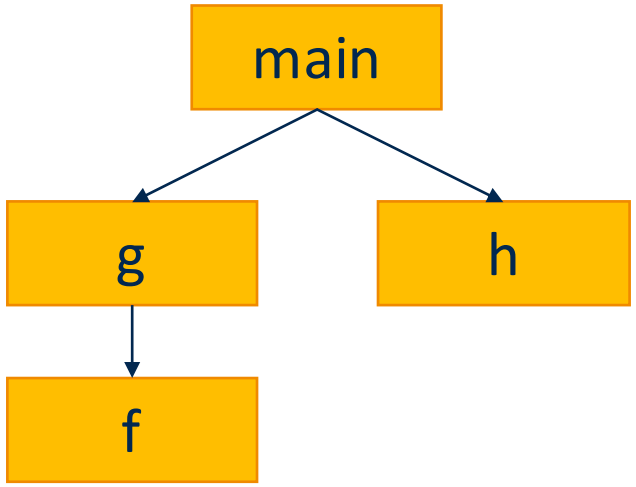
Call graph

- A call graph is a directed graph where:
  - Nodes represent functions
  - Edges represent calls — an edge from A to B means function A calls function B

```
void f(int a, int b) {
    // -- snip
}
void g(int a, b) { f(a, b); }
void h() { print("hello!"); }

int main(int argc, char* argv[]) {
    int i = atoi(argv[1]);
    if (i > 100) {
        g(i, 400-i);
    } else {
        h();
    }
}
```

Call graph

Example: callgraph

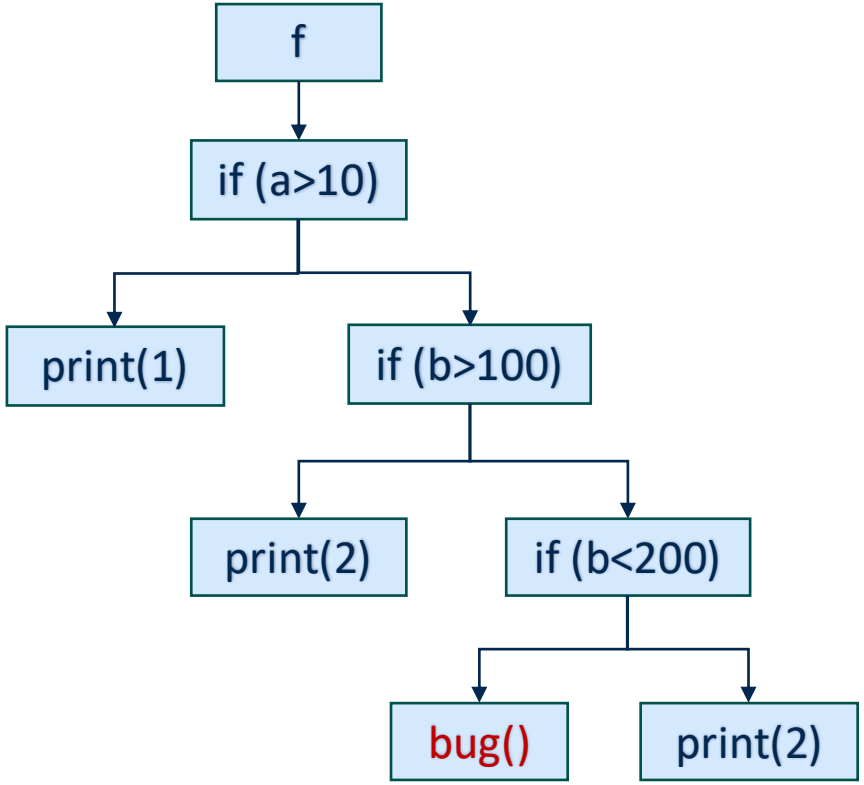


Call graph

```
void f(int a, int b) {
    // -- snip
}
void g(int a, b) { f(a, b); }
void h() { print("hello!"); }

int main(int argc, char* argv[]) {
    int i = atoi(argv[1]);
    if (i > 100) {
        g(i, 400-i);
    } else {
        h();
    }
}
```

Control flow graph

CFG for function f

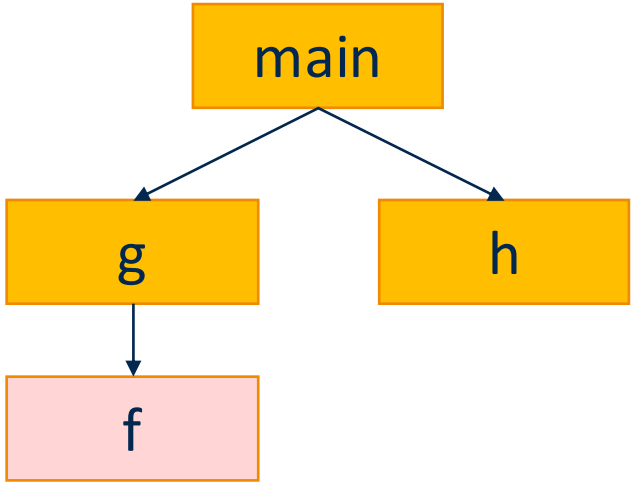


Control flow graph

```
void f(int a, int b) {
    if (a > 10) {
        print(1);
    } else {
        if (b > 100) {
            print(2);
        } else {
            if (b < 200) {
                bug();
            } else {
                print(3);
            }
        }
    }
}
```

DGF process

- Identify target functions in CG

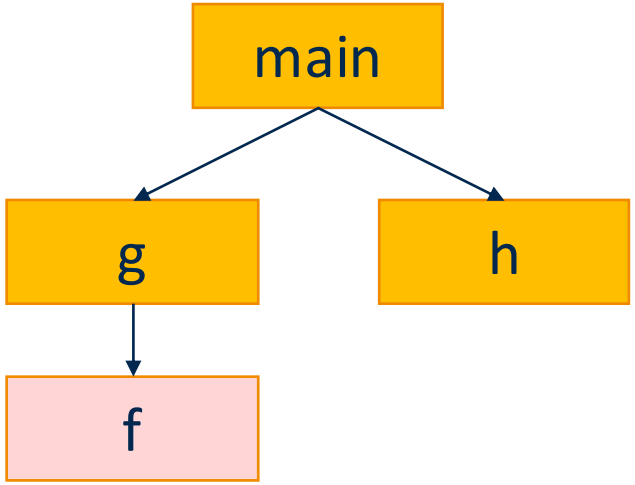


Call graph

```
void f(int a, int b) {
    // -- snip
}
void g(int a, b) { f(a, b); }
void h() { print("hello!"); }

int main(int argc, char* argv[]) {
    int i = atoi(argv[1]);
    if (i > 100) {
        g(i, 400-i);
    } else {
        h();
    }
}
```

DGF process

- Identify target functions in CG
- For each function, compute the length of the shortest path to target



Call graph

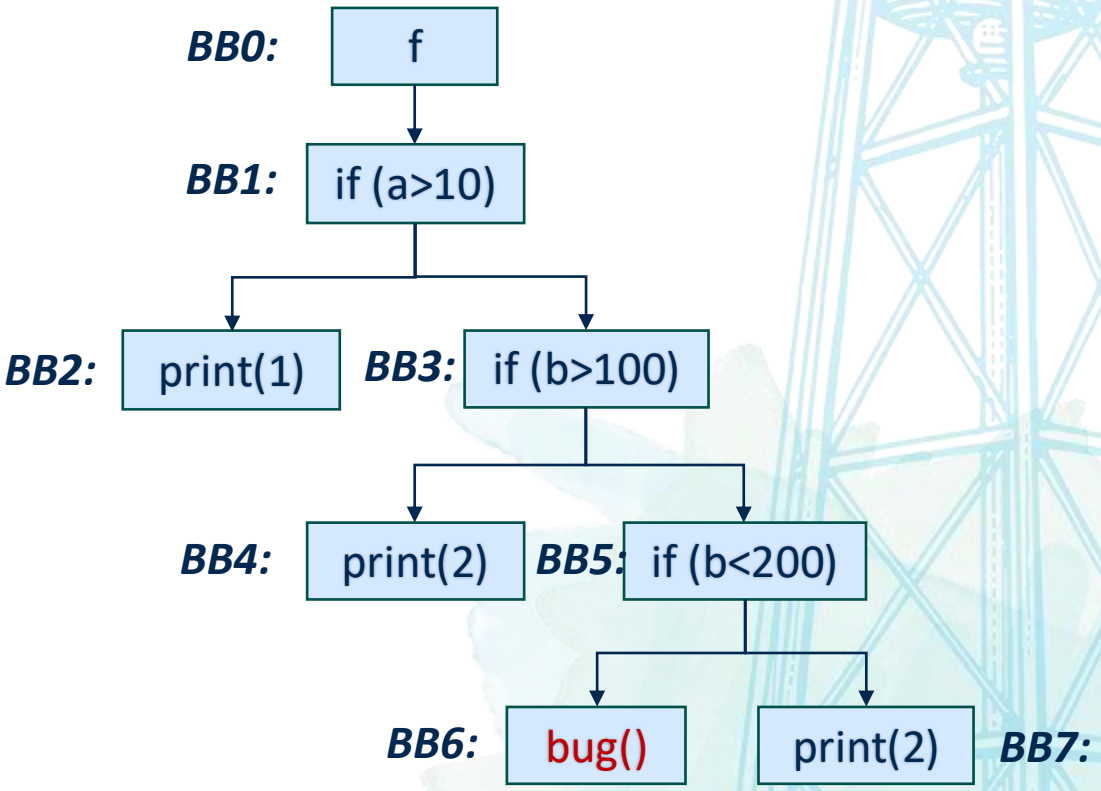
```
void f(int a, int b) {
    // -- snip
}
void g(int a, b) { f(a, b); }
void h() { print("hello!"); }

int main(int argc, char* argv[]) {
    int i = atoi(argv[1]);
    if (i > 100) {
        g(i, 400-i);
```

Function	Distance
main	2
g	1
h	Inf
f	0

DGF process

For the function containing the target, compute the BB distances to the target BB

Basic block	Distance
BB0	4
BB1	3
BB2	Inf
BB3	2
BB4	Inf
BB5	1
BB6	0
BB7	Inf



Control flow graph

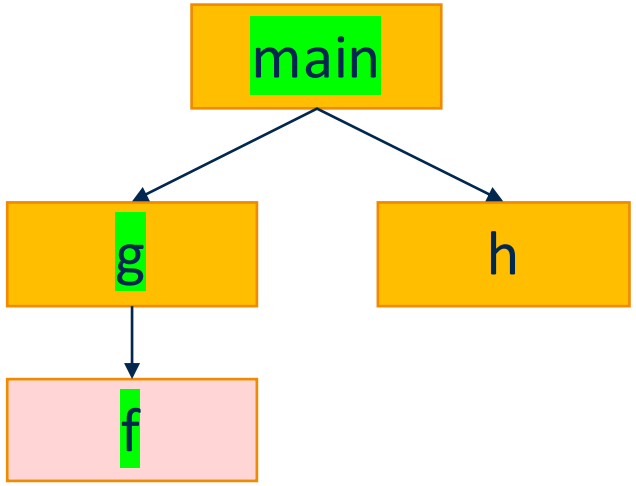
DGF process

- After each run, fuzzer compares the BBs executed for the shortest distance to the target
- Keeps seeds with lower distance
- Discards seeds with higher distance
- Executions gradually progress towards the target
- Example: AFLGo

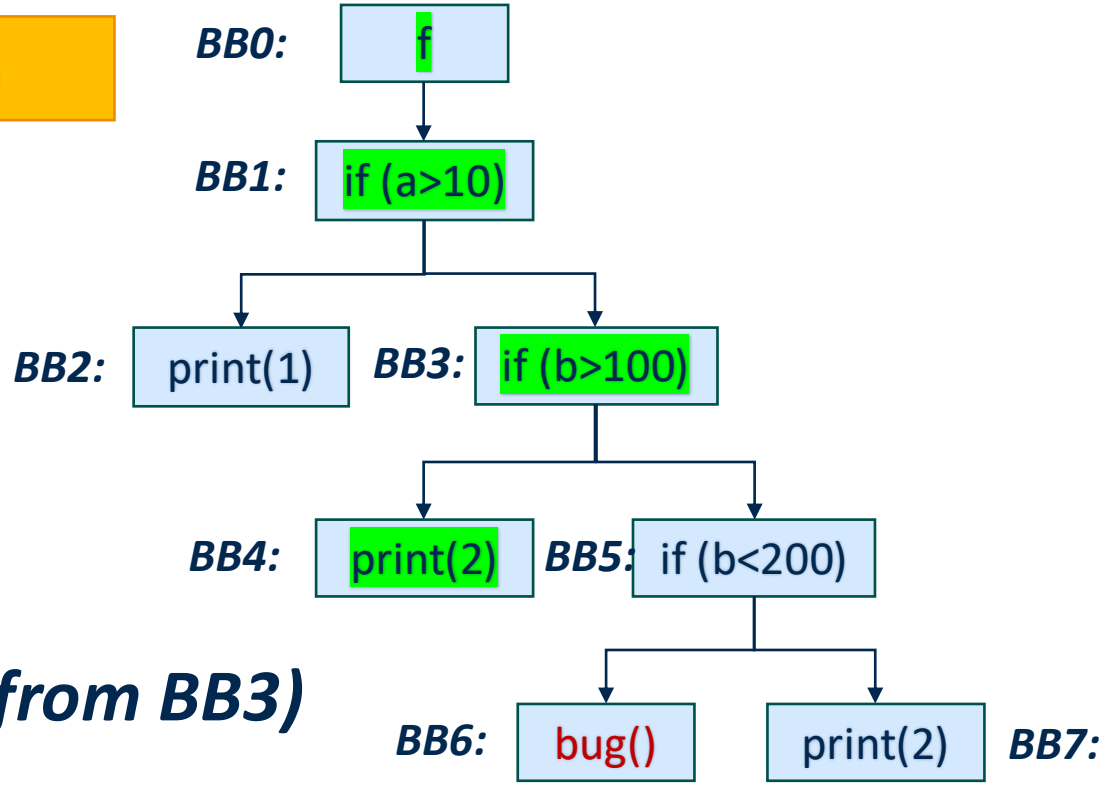
Function	Distance
main	2
g	1
h	Inf
f	0

Basic block	Distance
BB0	4
BB1	3
BB2	Inf
BB3	2
BB4	Inf
BB5	1
BB6	0
BB7	Inf

Sample coverage for input



Call graph



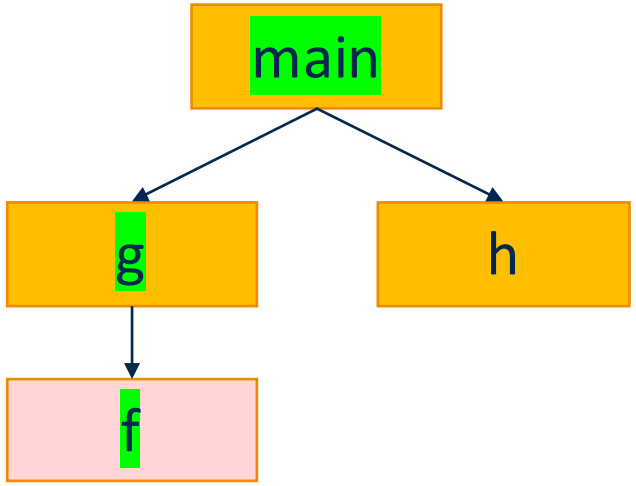
Control flow graph

Min. distance: 2 (from BB3)

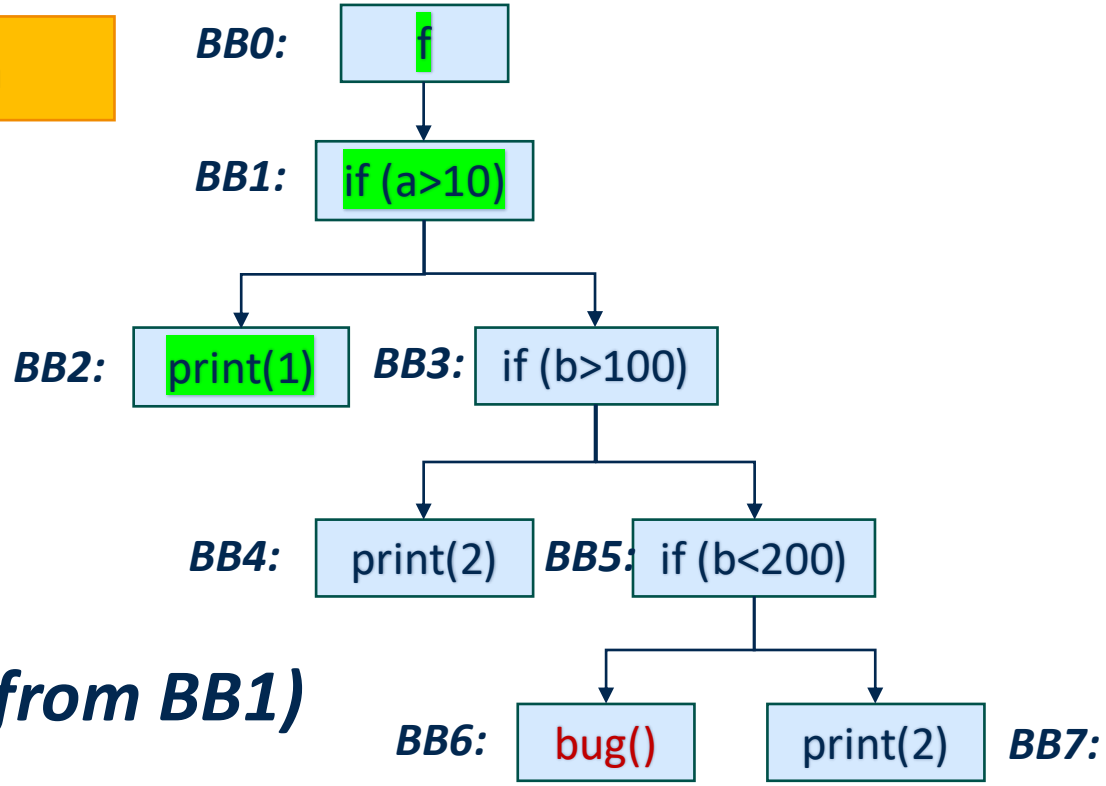
Function	Distance
main	2
g	1
h	Inf
f	0

Basic block	Distance
BB0	4
BB1	3
BB2	Inf
BB3	2
BB4	Inf
BB5	1
BB6	0
BB7	Inf

Sample coverage for input



Call graph



Control flow graph

Function	Distance
main	2
g	1
h	Inf
f	0

Basic block	Distance
BB0	4
BB1	3
BB2	Inf
BB3	2
BB4	Inf
BB5	1
BB6	0
BB7	Inf

Min. distance: 3 (from BB1)
→ discard seed!

Fuzz testing - sanitizers

Tapti Palit

Recall: memory safety bugs

- Fuzzers execute the program with random inputs and look for crashes
- But memory safety bugs may or may not crash the program
- **Undefined Behavior (UB)**

UB example

Buffer overflow can silently corrupt the `isAdmin` variable

```
int main(int argc, char** argv) {
    int isAdmin = 0;
    char arr[4];

    int index = atoi(argv[1]);

    arr[index] = ‘A’;
    // --snip
}
```

Sanitizers: oracles for memory safety bugs

- Adds additional compiler instrumentation to record object metadata
- Turns silent UBs to crashes
- Popular sanitizers
  - Address sanitizer (ASAN) – can detect buffer overflow
  - Leak sanitizer (LSAN) – can detect memory leak
  - Thread sanitizer (TSAN) – can detect data races
- ***Significantly*** slows down the application; but can detect memory safety bugs!!

Sanitizer instrumentation

- Sanitizers insert additional instrumentation
- Implemented as an additional compiler pass in `afl-clang`
- `afl-clang –fsanitize=address x.c –o x.o`

Address sanitizer: high level idea

- Goal – detect buffer overflow
- During program execution, what information is needed to determine if `function()` causes a buffer overflow?

```
void function(char* p) {
    p[0] = ‘h’;
    p[1] = ‘e’;
    p[2] = ‘l’;
    p[3] = ‘l’;
    p[4] = ‘o’;
    p[5] = ‘\n’;
}

int main(void) {
    char* s = malloc(4);
    function(s);
}
```

Address sanitizer: high level idea

- Address sanitizer is a compiler tool
- When compiling the program, it adds code to
  - Record the size of heap buffers
  - Check if an out-of-bounds access occurs

```
void function(char* p) {
    p[0] = 'h';
    p[1] = 'e';
    p[2] = 'l';
    p[3] = 'l';
    p[4] = 'o';
    p[5] = '\n';
}

int main(void) {
    char* s = malloc(4);
    function(s);
}
```

Check if each offset is within the heap object size

Must record the size of the allocated heap object

Metadata recording approaches

- Option 1: maintain fine-grained metadata for each object
  - Maintain a HashMap `{obj_addr: len}`
- Instrument heap allocation with HashMap insertion
- Instrument pointer dereference with HashMap lookup

```
void function(char* p, char q) {
    p[0] = ‘h’;
    q[1] = ‘e’;
}

int main(void) {
    char* s = malloc(4);
    char* t = malloc(8);
    function(s, t);
}
```

Metadata recording approaches

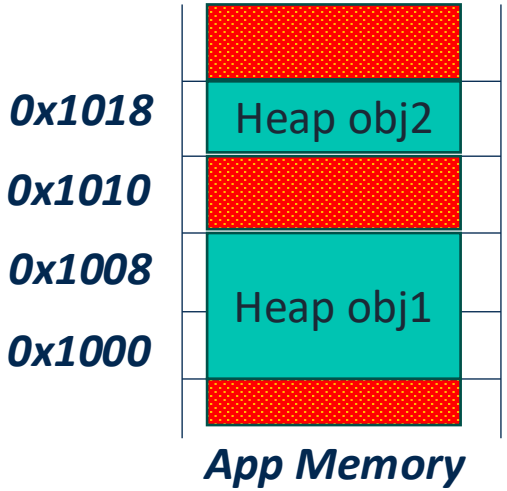
- Option 1: maintain fine-grained metadata for each object
  - Maintain a HashMap `{obj_addr: len}`
- Instrument heap allocation with HashMap insertion
- Instrument pointer dereference with HashMap lookup
- ***... very expensive!!!***

```
Map<void*, int> objMap;

void function(char* p, char q) {
    int obj_len = lookup_hashmap(p);
    if ( p + 0 > p + obj_len) assert(false);
    p[0] = 'h';
    obj_len = lookup_hashmap(q);
    if ( q + 1 > q + obj_len) assert(false);
    q[1] = 'e';
}

int main(void) {
    char* s = malloc(4);
    record_object_in_hashmap(s);
    char* t = malloc(8);
    record_object_in_hashmap(t);
    function(s, t);
}
```

ASan

- ASan is a sanitizer used with popular fuzzers (AFL, libfuzzer)
  - Goal: low overhead
- Insert “red/poison zones” around memory
- Updates to red zones indicate buffer overflow

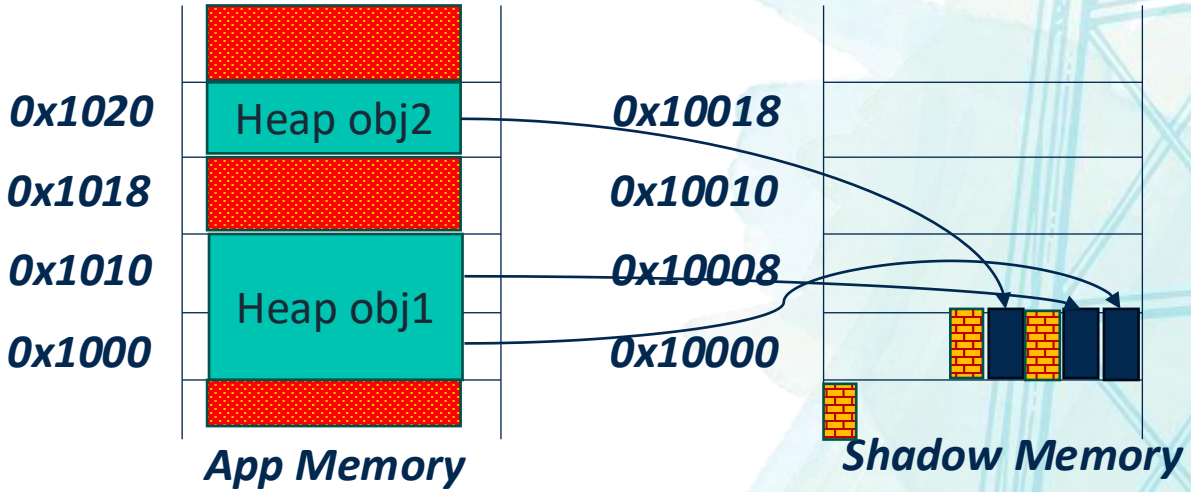
```
void function(char* p) {
    while(*p != ‘\0’) *p++ = ‘a’;
}

int main(void) {
    char* s = malloc(16); // --snip
    char* p = malloc(8); // --snip
    function(s);
}
```



Shadow memory

- Maintains shadow memory to record metadata about red and accessible zones
- Each ***byte*** in shadow memory represents a corresponding ***8 bytes*** in app
  - 0x00 for valid
  - 0xFF for poisoned range
  - 0x01 – 0x07 – partially poisoned

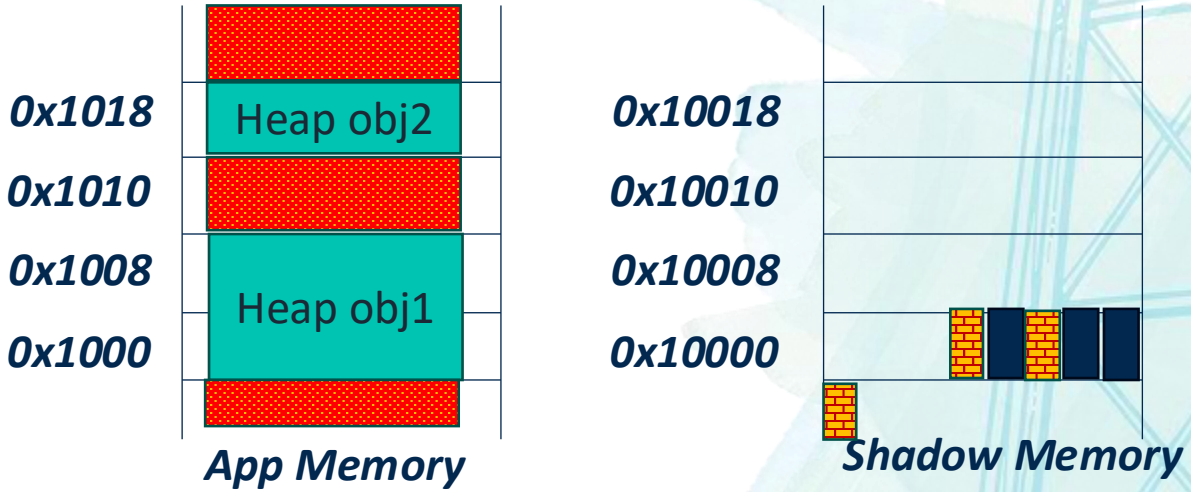
```
void function(char* p) {
     while(*p != '\0') *p++ = 'a';
}


int main(void) {
    char* s = malloc(16); // --snip
    char* p = malloc(8); // --snip
    function(s);
}
```



Compiler checks

- Compiler adds instructions before each memory access to first check the shadow memory byte

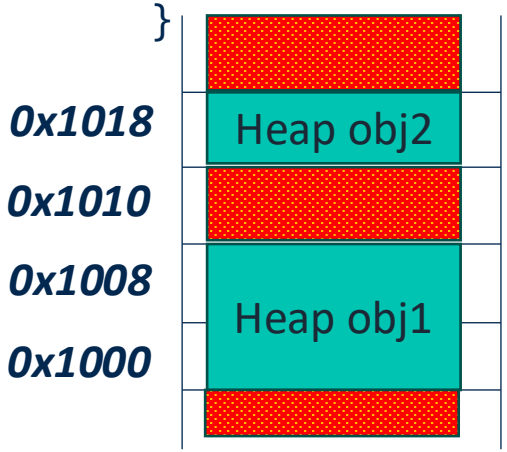
```
void function(char* p) {
     while(*p != '\0') *p++ = 'a';
}


int main(void) {
    char* s = malloc(16); // --snip
    char* p = malloc(8); // --snip
    function(s);
}
```



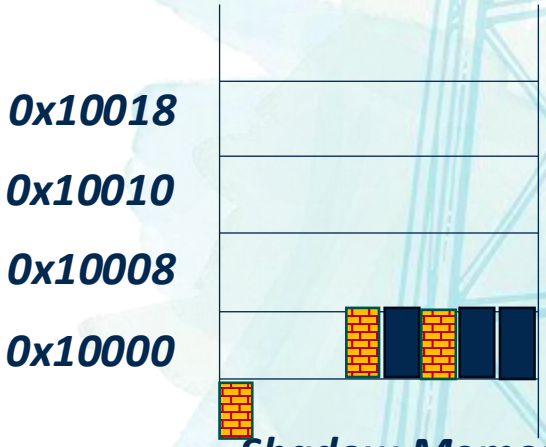
Compiler checks pseudocode

- Compiler adds instructions before each memory access to first check the shadow memory byte
- Pseudocode
  - shdw_poisoned(void*) computes the address of the shadow byte
  - Checks if it is fully/partially poisoned
- ***Question: can ASan detect all buffer overflows?***

```
void function(char* p) {
     while(*p != '\0') {
        if (shdw_poisoned(p)) { assert(false); }
        *p++ = 'a';
     }
}

int main(void) {
    char* s = malloc(16); // heap object 1
    char* p = malloc(8); // heap object 2
    function(s);
}
```



App Memory



Shadow Memory

Why shadow memory?

- Shadow memory stores metadata about the 8-byte memory range
- Fast computation of shadow byte address
  - `shadow_addr = (address >> 3) + SHADOW_MEMORY_BASE`
- Fast checks of shadow byte value
  - `shadow_value = *shadow`
- Alternatively, complex data-structures would be very expensive

Fuzzer success factors

- Fuzzer throughput (exec/s)
- Percentage of valid inputs generated by the mutator
- Type of coverage recorded
- Sanitizer vs. no sanitizer
- Ideally, want a high percentage of valid inputs + fine-grained code and data coverage + sanitizers
  - Unfortunately reduces throughput
- Tradeoff: finding the right balance for your use case!

Summary

- Fuzzing is an effective testing mechanism
- Unlike other testing approaches it can ***find new inputs*** that cause program misbehavior
- Mutation and coverage are important for generating effective fuzzers
- Directed greybox fuzzing can target specific parts of the codebase
- Sanitizers provide additional information needed to find *silent UB* bugs

Symbolic Execution

Tapti Palit

Acknowledgement: Suman Jana (Columbia Univ.) Baishakhi Ray (UVA), Omar Chowdhury (Purdue), Saswat Anand (GA Tech), Rupak Majumdar (UCLA), Koushik Sen (UCB)

UCDAVIS

Limitations of fuzzing

- Fuzzing generates inputs randomly or by mutation — effective but incomplete
- Struggles with:
  - Magic constants — if (x == 0xDEADBEEF) is almost impossible to hit randomly
  - Deep paths — reaching code guarded by many nested conditions
  - Checksums and hash comparisons — random bytes rarely pass validation
- Can we systematically generate inputs that ***comprehensively*** explores different program paths?

Background

SAT and SMT Solvers

SAT: boolean satisfiability

- Given a propositional formula in CNF, find a satisfying assignment
- CNF: conjunction of clauses, each clause is a disjunction of literals
  - Example: $(b \vee c) \wedge (\neg a \vee d) \wedge (\neg b \vee d)$
  - Assignment: {a=FALSE, b=TRUE, c=FALSE, d=TRUE} → formula is TRUE
- SAT is NP-complete — worst case exponential
  - But modern SAT solvers (DPLL, CDCL) handle millions of variables in practice
- SAT solvers are the foundation for SMT solvers

SMT: Satisfiability Modulo Theories

- SMT generalizes SAT: Boolean variables are replaced by predicates over richer theories
- Input: a first-order formula over background theories
- Output: is the formula satisfiable? If so, provide a model (variable assignments)
- Common theories used in program analysis:
  - Arithmetic — linear and nonlinear integer/real arithmetic
  - Arrays — read/write operations on arrays
  - Bit-vectors — fixed-width binary arithmetic (models machine integers exactly)
  - Uninterpreted Functions — function equality: `f(a) = f(b) if a = b`

SMT example for arithmetic

- Formula: $x + 2y = 10 \wedge x > y$
  - x, y are integer variable and not just boolean variables
  - Supports arithmetic operations
  - Is this formula satisfiable?
    - Yes!
- What is a satisfiable assignment?
  - $x = 8, y = 1$
  - $x = 6, y = 2$
  - $x = 4, y = 3$

SMT example for array

- Formula: `read(write(A, 3, 42), i) = 42 ∧ i + 1 = 4`
- `write(A, 3, 42) -> A'`
  - `A'[x] = A[x]` where `x != 3`
  - `A'[x] = 42` where `x == 3`
- Is this formula satisfiable?
  - Yes!
- What is a satisfiable assignment?
  - `i = 3`

SMT solvers

Popular SMT solvers: Z3 (Microsoft), STP, and many more

$x + 2y = 10 \land x > y$

```
read(write(A, 3, 42),
i) = 42 ∧ i + 1 = 4
```

$x = 8, y = 1$

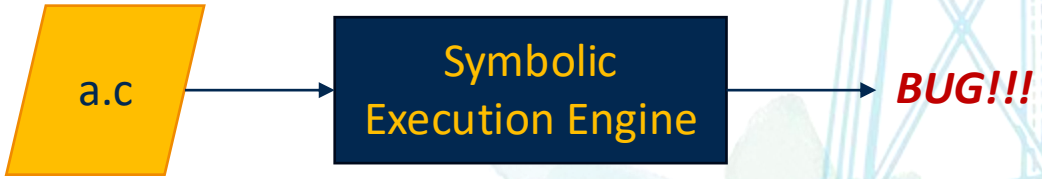
$i = 3$

What is symbolic execution?

- Execute the program with symbolic inputs instead of concrete values
- Instead of $x = 5$, we have $x = \alpha$ (a symbolic variable)
- As execution proceeds:
  - Variables hold symbolic expressions (e.g., $z = 2\alpha$)
  - At each branch, the engine forks into two paths
  - Each path accumulates a path constraint (conjunction of branch conditions)

What is symbolic execution?

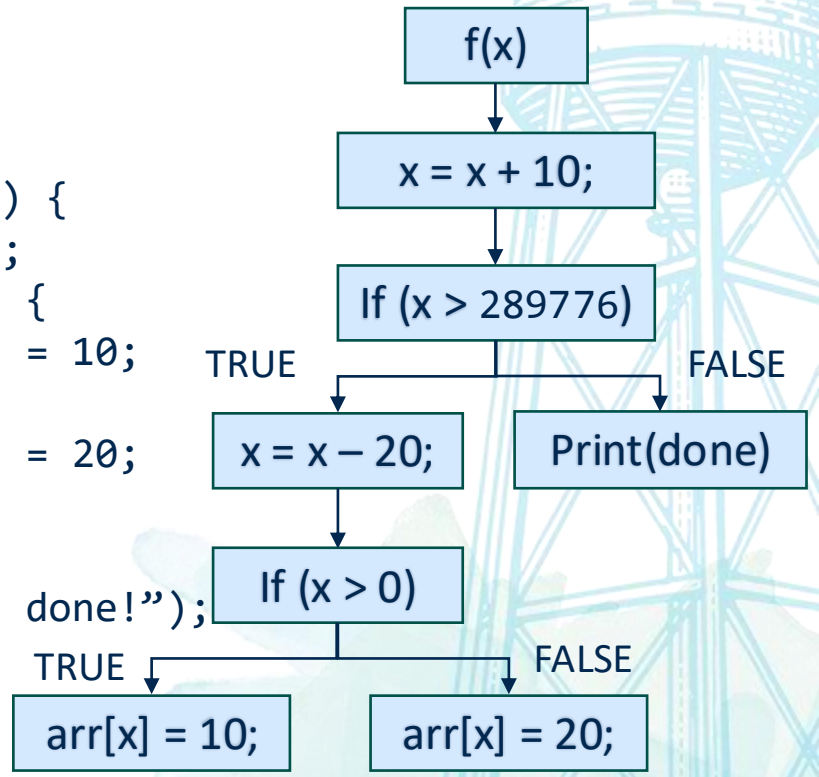
- Execute the program with symbolic inputs instead of concrete values
- Instead of x = 5, we have x = α (a symbolic variable)
- As execution proceeds:
  - Variables hold symbolic expressions (e.g., z = 2α)
  - At each branch, the engine forks into two paths
  - Each path accumulates a path constraint (conjunction of branch conditions)



What is symbolic execution?

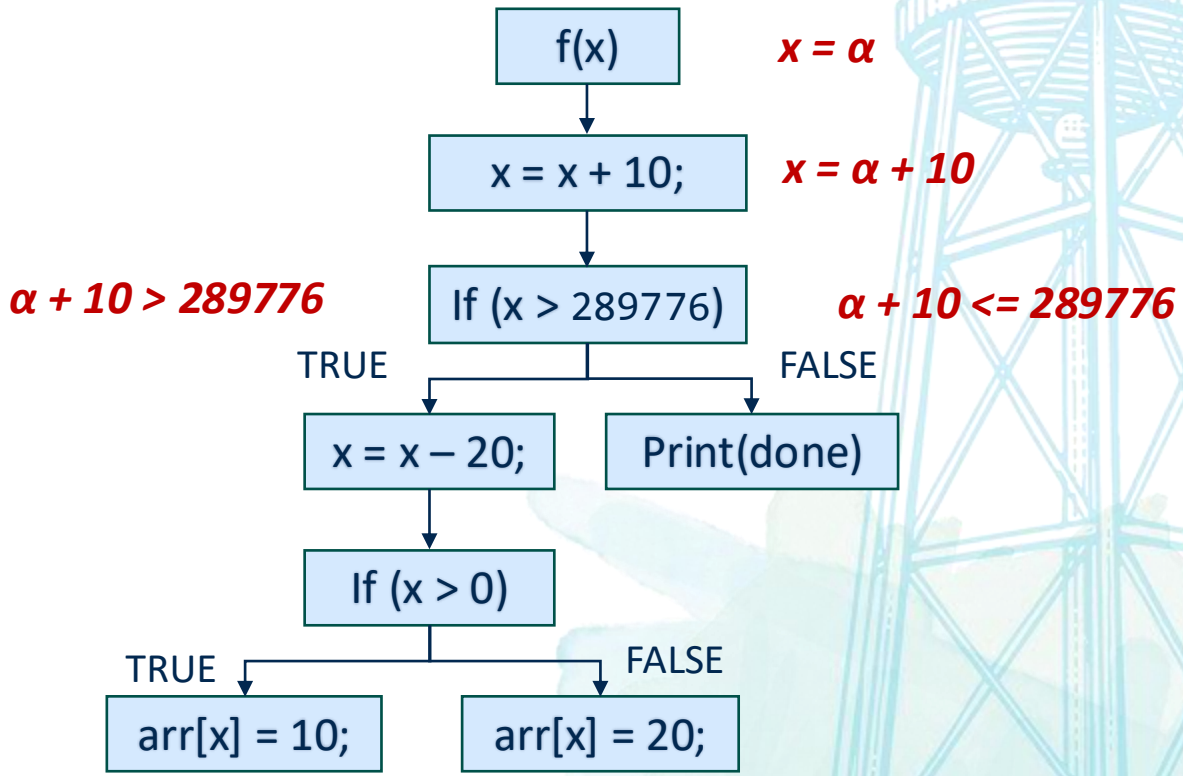
- Execute the program with symbolic inputs instead of concrete values
- Instead of x = 5, we have x = α (a symbolic variable)
- As execution proceeds:
  - Variables hold symbolic expressions (e.g., z = 2α)
  - At each branch, the engine forks into two paths
  - Each path accumulates a path constraint (conjunction of branch conditions)

```
void f(int x) {
    int arr[10];
    x = x + 10;
    if (x > 289776) {
        x = x - 20;
        if (x > 0) {
            arr[x] = 10;
        } else {
            arr[x] = 20;
        }
    } else {
        print("all done!");
    }
}
```



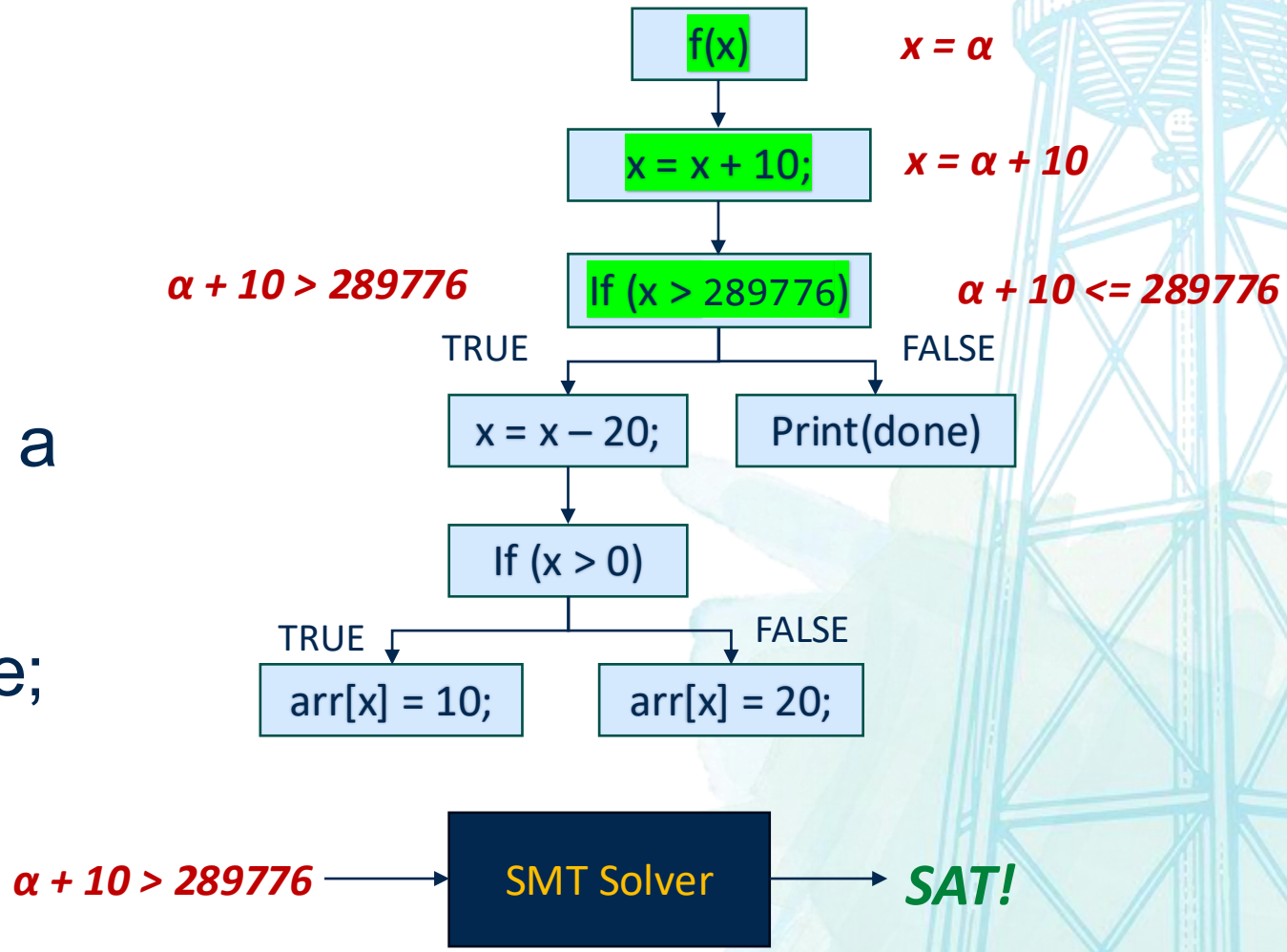
What is symbolic execution?

- Execute the program with symbolic inputs instead of concrete values
- Instead of x = 5, we have x = α (a symbolic variable)
- As execution proceeds:
  - Variables hold symbolic expressions (e.g., z = 2α)
  - At each branch, the engine forks into two paths
  - Each path accumulates a path constraint (conjunction of branch conditions)



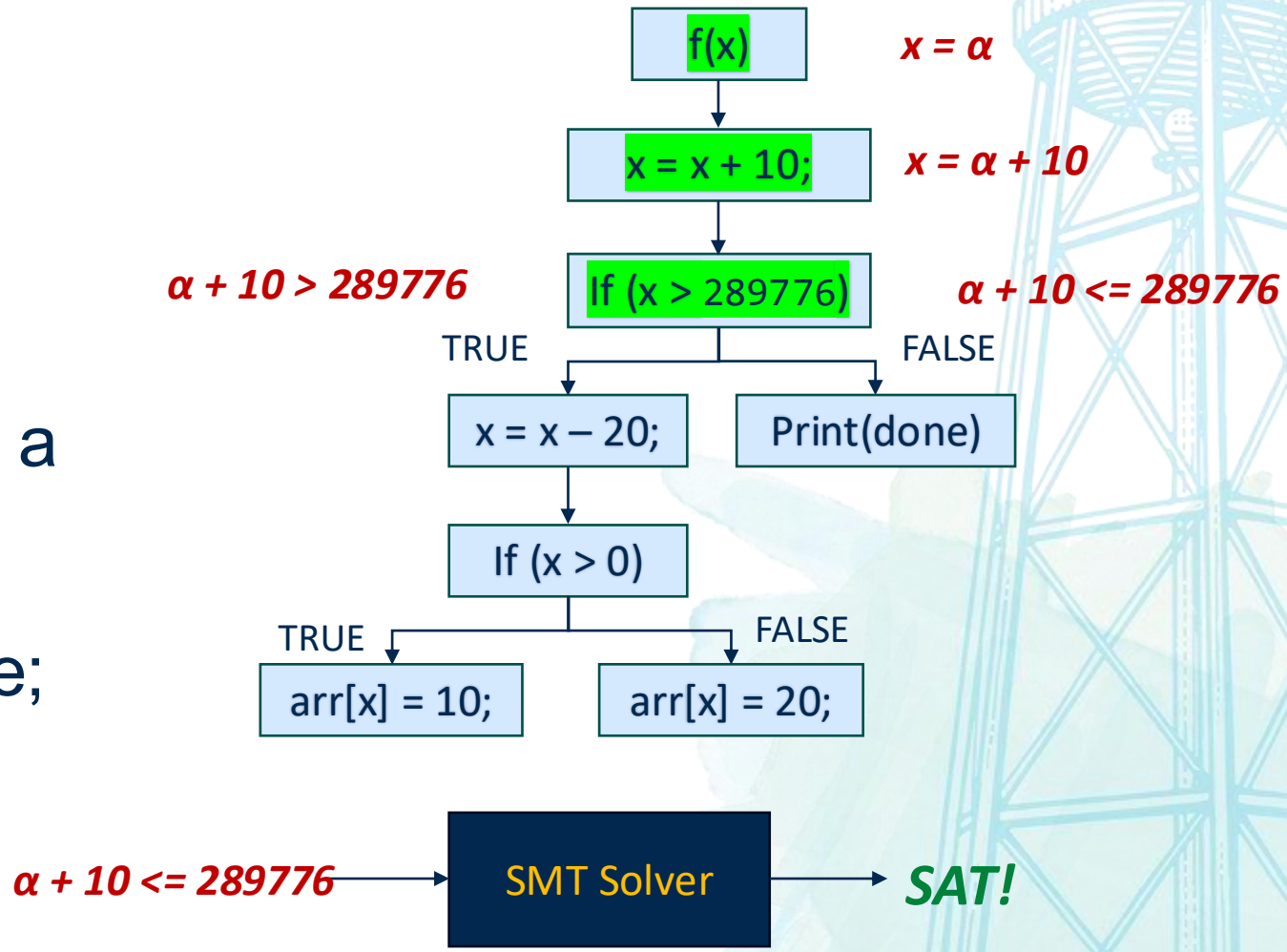
Path constraint

- Path constraint: quantifier-free formula encoding all branch decisions so far
  - Example: $2\beta = \alpha \wedge \alpha > \beta + 10$
- Solve the path constraint → get a concrete test input for that path
- All paths form the execution tree; some are feasible, some are infeasible



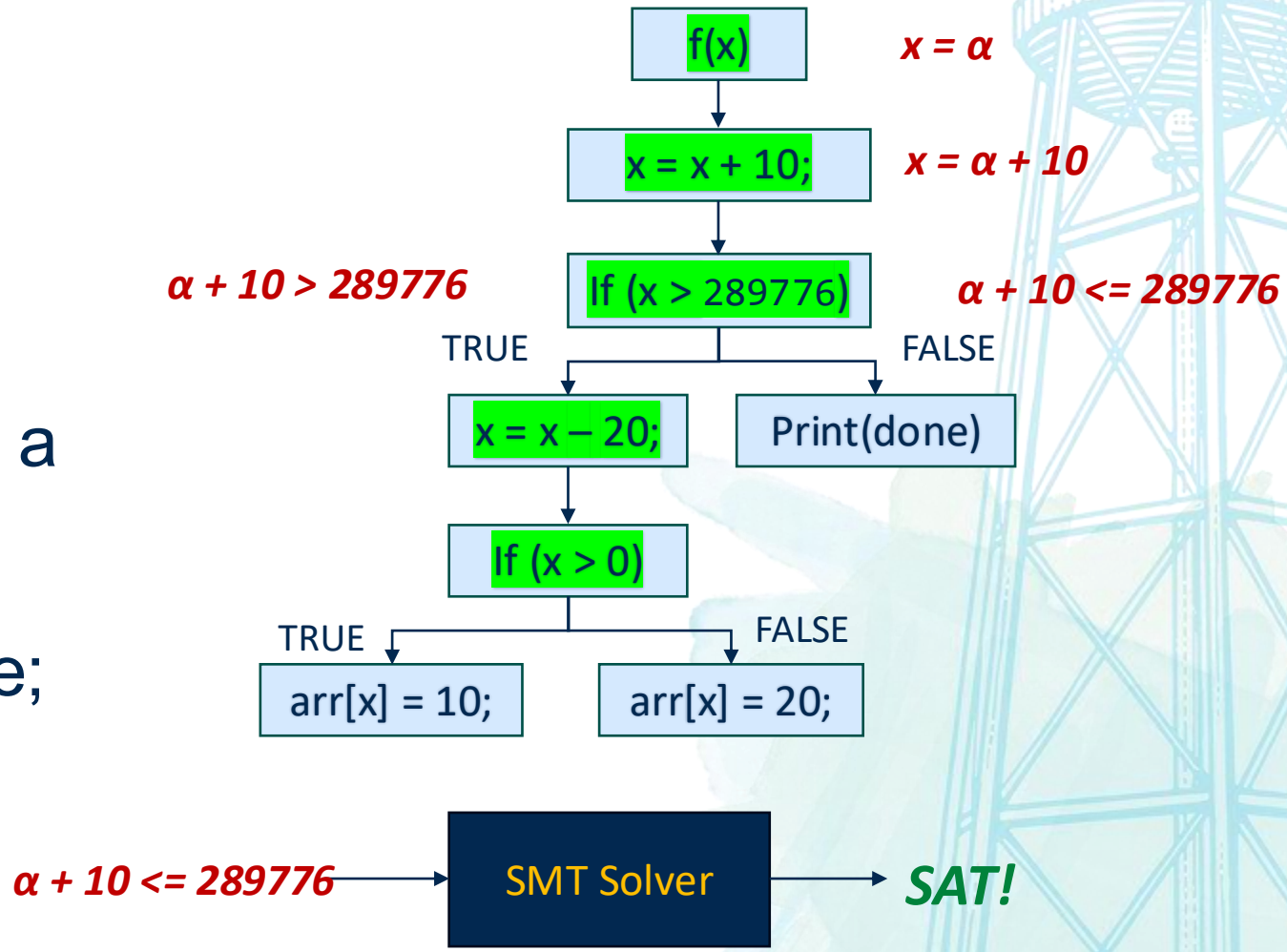
Path constraint

- Path constraint: quantifier-free formula encoding all branch decisions so far
  - Example: $2\beta = \alpha \wedge \alpha > \beta + 10$
- Solve the path constraint → get a concrete test input for that path
- All paths form the execution tree; some are feasible, some are infeasible



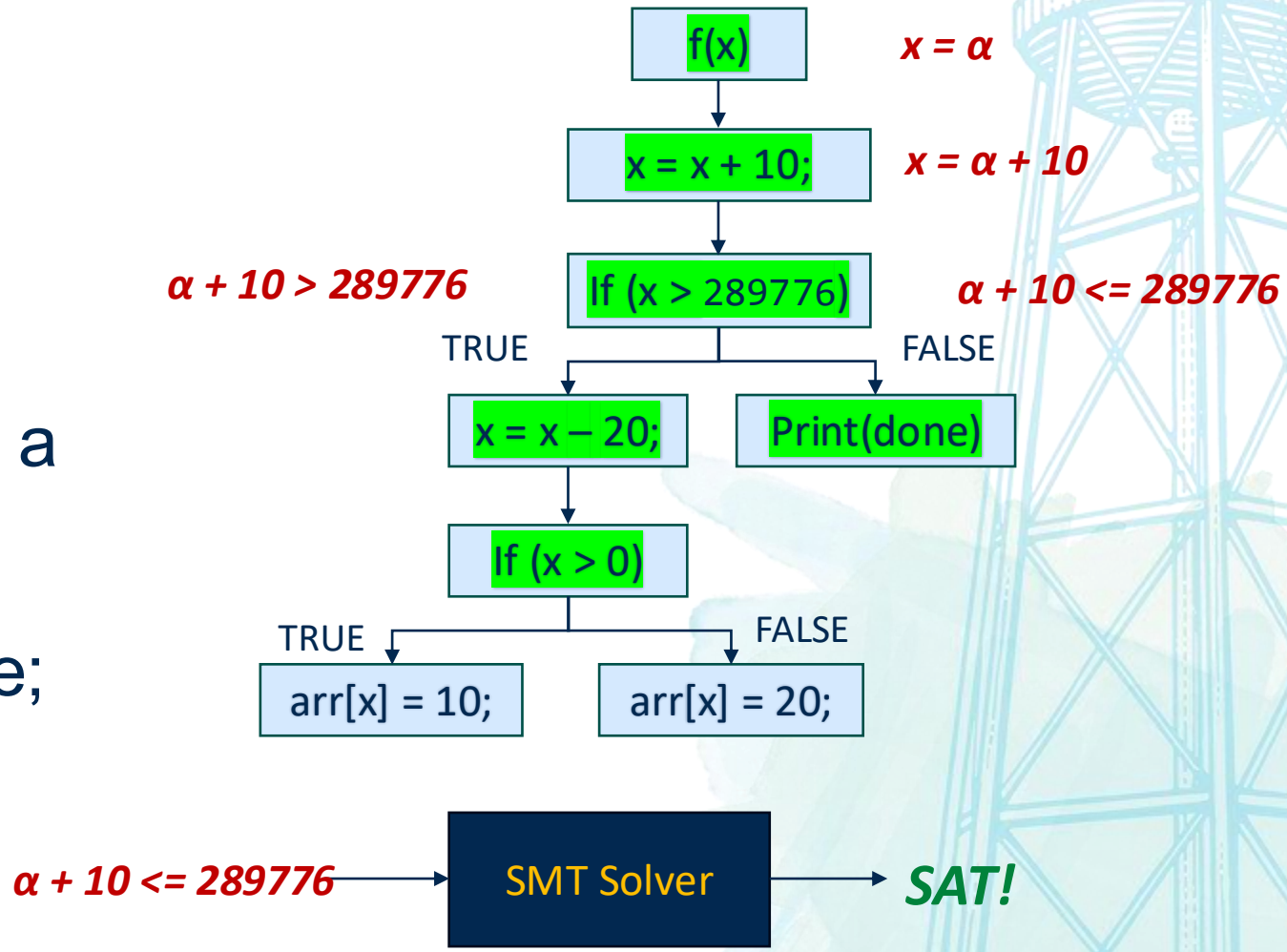
Path constraint

- Path constraint: quantifier-free formula encoding all branch decisions so far
  - Example: $2\beta = \alpha \wedge \alpha > \beta + 10$
- Solve the path constraint → get a concrete test input for that path
- All paths form the execution tree; some are feasible, some are infeasible



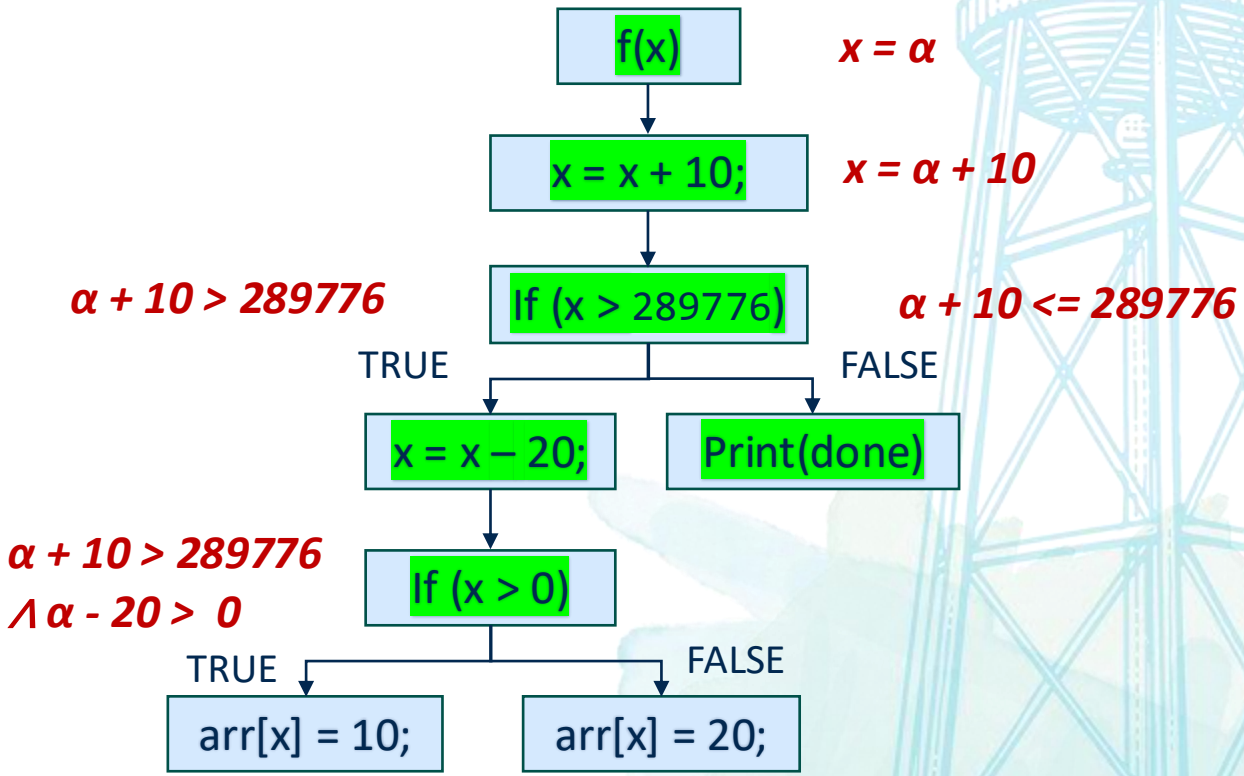
Path constraint

- Path constraint: quantifier-free formula encoding all branch decisions so far
  - Example: $2\beta = \alpha \wedge \alpha > \beta + 10$
- Solve the path constraint → get a concrete test input for that path
- All paths form the execution tree; some are feasible, some are infeasible



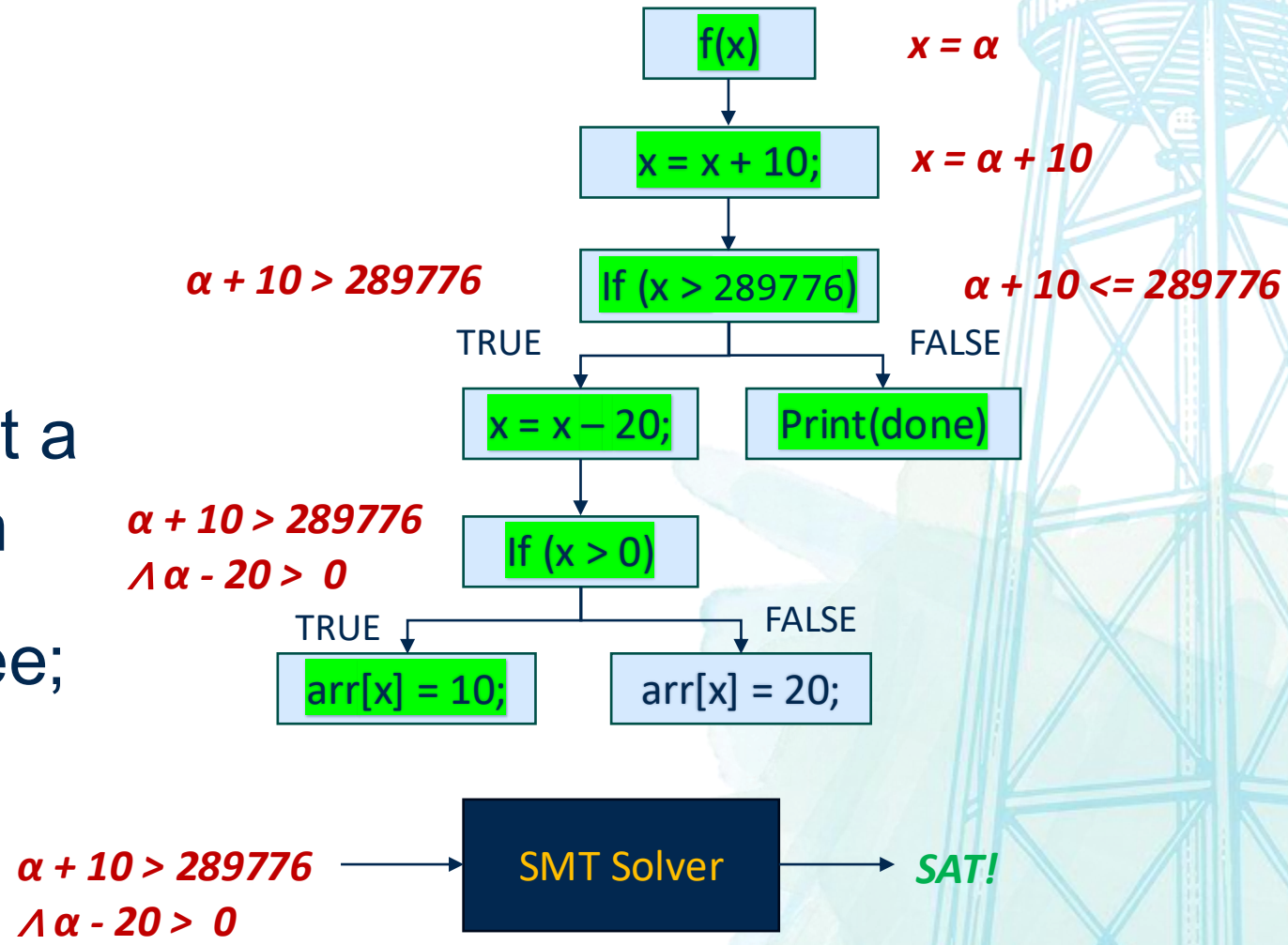
Path constraint

- Path constraint: quantifier-free formula encoding all branch decisions so far
  - Example: $2\beta = \alpha \wedge \alpha > \beta + 10$
- Solve the path constraint → get a concrete test input for that path
- All paths form the execution tree; some are feasible, some are infeasible



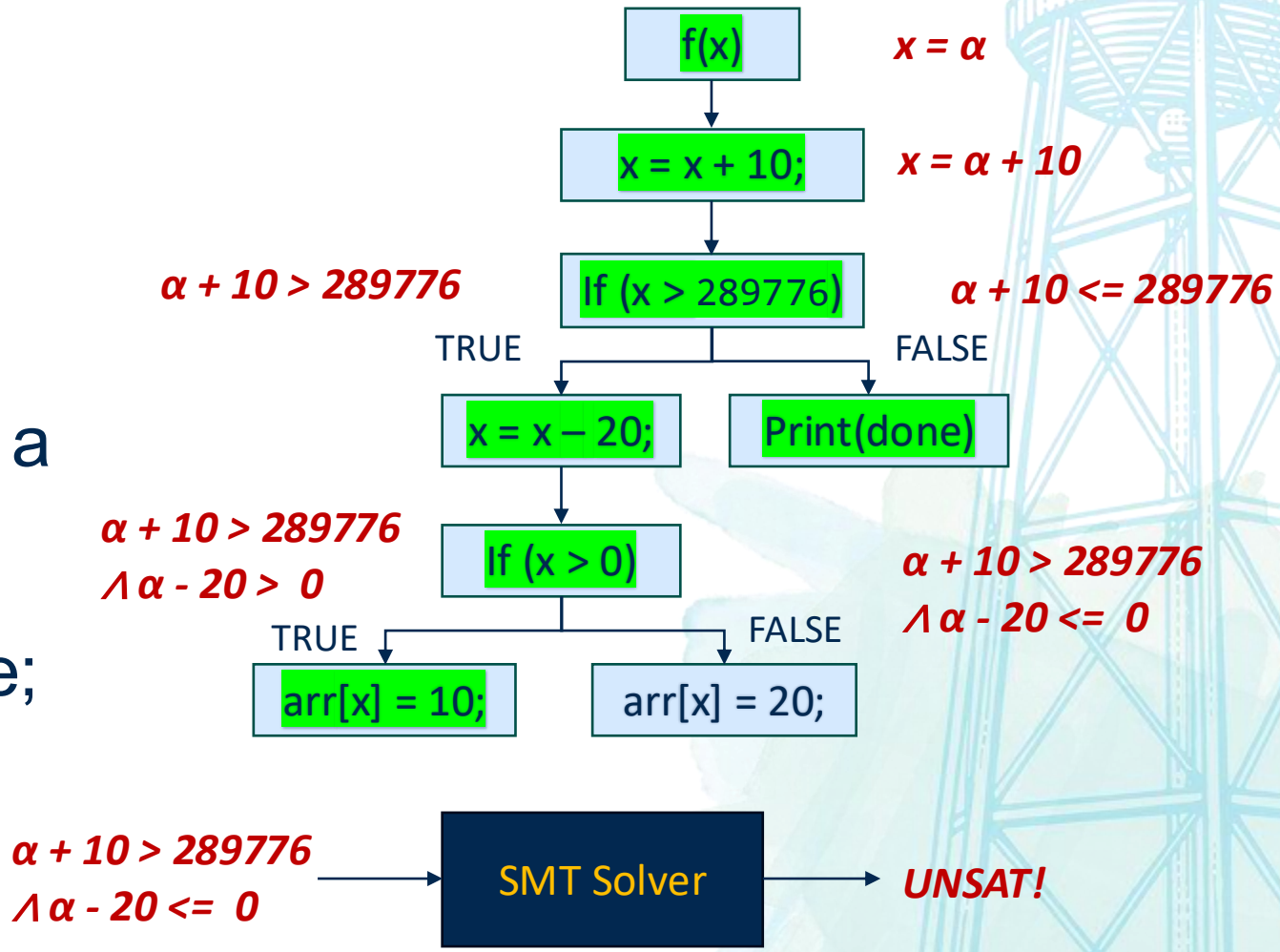
Path constraint

- Path constraint: quantifier-free formula encoding all branch decisions so far
  - Example: $2\beta = \alpha \wedge \alpha > \beta + 10$
- Solve the path constraint → get a concrete test input for that path
- All paths form the execution tree; some are feasible, some are infeasible



Path constraint

- Path constraint: quantifier-free formula encoding all branch decisions so far
  - Example: $2\beta = \alpha \wedge \alpha > \beta + 10$
- Solve the path constraint → get a concrete test input for that path
- All paths form the execution tree; some are feasible, some are infeasible



Symbolic execution workflow

- Workflow:
  - Mark inputs as symbolic
  - Engine executes symbolically
  - At branch: query solver
  - Fork into feasible paths
  - At path end: solve constraint
  - Solver returns concrete input (test case)
- Implementations: KLEE, S2E

```
#include <klee/klee.h>

int main() {
    int x;
    klee_make_symbolic(&x, sizeof(x), "x");
     // mark x as symbolic

    if (x > 0) {
        if (x < 100) {
            printf("Small positive\n");
        } else {
            printf("Large positive\n");
        }
    } else {
        printf("Non-positive\n");
    }
    return 0;
}

// KLEE generates 3 test cases:
//   test1: x = -1  (non-positive path)
//   test2: x = 50  (small positive path)
//   test3: x = 200 (large positive path)
```

Symbolic vs. concrete execution

Concrete Execution	Symbolic Execution
Variables hold concrete values E.g. $x = 5$, $y = 3$, $z = 6$	Variables hold symbolic expressions $x = \alpha$, $y = \beta$, $z = 2\beta$
One path through the program	Explores all paths
Tests one input at a time	Each path = equivalence class of inputs
Fast — native execution speed	Slower — needs SMT solver queries
E.g. fuzzing, unit testing	Symbolic execution known as ***whitebox testing***

How symbolic execution finds bugs

- Symbolic execution doesn't just explore paths; it can check for bugs
- For each dangerous operation, insert a check:
  - Division: fork on `divisor == 0`
  - Array access: fork on `index < 0` or `index ≥ length`
  - Pointer dereference: fork on `pointer == NULL`
  - Assertion `assert(P)`: fork on ¬P
  - Integer overflow: fork on result exceeding type bounds

How symbolic execution finds bugs

- If the "bug" branch is satisfiable → solver provides a concrete bug-triggering input
- If UNSAT → the bug cannot happen on this path (proven safe!)

Example: division by zero checker

- At $y = x / z$ where z is symbolic and current path constraint is PC:
  - Fork into two paths — one where $z = 0$ and one where $z \neq 0$
  - Solving $PC \wedge z = 0$ gives a concrete input triggering the bug

Example: division by zero checker

- At y = x / z where z is symbolic and current path constraint is PC:
  - Fork into two paths — one where z = 0 and one where z ≠ 0
  - Solving PC ∧ z = 0 gives a concrete input triggering the bug

```
int compute(int x, int z) {

    klee_make_symbolic(z, sizeof(int));

    // many operations involving z

    if (x > 0) {
        int y = x / z; // dangerous op
        return y;
    }
    return 0;
}
```

Example: division by zero checker

- Assume PC is $\beta < 4 \wedge \gamma > 0$
- Assume symbolic value of z is $\alpha + 2\beta - 20$

```
int compute(int x, int z) {

    klee_make_symbolic(z, sizeof(int));

    // many operations involving z

    if (x > 0) {
        int y = x / z; // dangerous op
        return y;
    }
    return 0;
}
```

$(\beta < 4 \wedge \gamma) > 0 \wedge (\alpha + 2\beta - 20) = 0$ → SMT Solver → *SAT*

Div by zero detected!

Example: division by zero checker

- Assume PC is $\beta < 4 \wedge \gamma > 0$
- Assume symbolic value of z is $\alpha + 2\beta - 20$

```
int compute(int x, int z) {

    klee_make_symbolic(z, sizeof(int));

    // many operations involving z

    if (x > 0) {
        int y = x / z; // dangerous op
        return y;
    }
    return 0;
}
```

$(\beta < 4 \wedge \gamma) > 0 \wedge (\alpha + 2\beta - 20) = 0$ → SMT Solver → *UNSAT!*

Div by zero not possible!

Challenges of Symbolic Execution

- Path explosion — N branches → up to 2^N paths; exponential blowup
- Loops and recursion; can produce infinite execution trees
- Heap and pointer modeling; symbolic addresses, aliasing analysis
- SMT solver limitations; floating point are hard/undecidable
- Environment modeling — system calls, file I/O, network, concurrency
- Symbolic execution works best on:
  - Smaller, well-bounded programs or functions
  - Programs with limited use of external libraries
  - Unit-level testing rather than whole-system testing

Summary

- Symbolic execution explores paths systematically using symbolic inputs + SMT solvers
- Path constraints define equivalence classes — one test input per class suffices
- KLEE: practical symbolic execution on LLVM bitcode;
- Challenges: path explosion, loops, environment modeling, solver limitations

Misc. Testing Topics

Continuous Integration

Continuous integration

- Code changes are automatically built, tested, and integrated into a shared repository multiple times a day
- Core process
  - Developer commits code
  - CI pipeline triggers
    - Build the project
    - Run automated tests
- Integrated into GitHub and GitLab
  - Github Actions

GitHub Actions

CI/CD

Recap: containerization

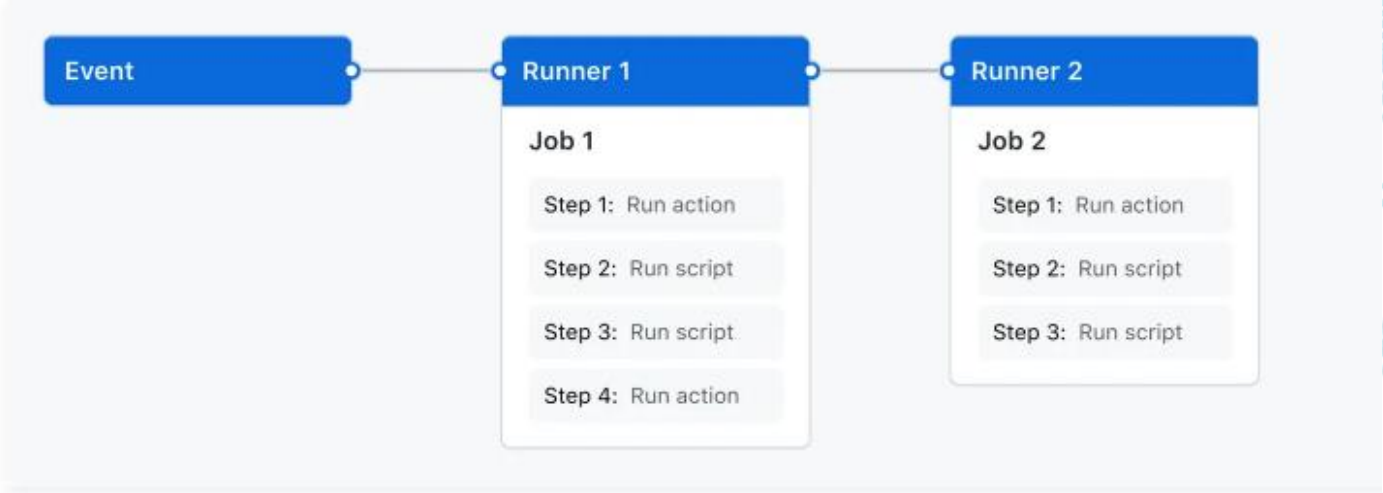
- Github Actions are executed on Docker containers
- Lightweight virtualization method that packages applications and their dependencies into isolated, portable containers
- Containers share the host OS kernel but have their own filesystem, resources, and dependencies

What do containers provide for CI?

- Uniform build system for continuous integration
  - No weird issues with `$PATH` variables
  - No missing binaries
- Automate deployment via continuous deployment
  - Containerize the application and deploy
  - … *More* l*ater*
- Solves ***Works-on-my-Machine*** issues

What is a Github Action?

- A GitHub Actions **workflow** to be triggered when an **event** occurs
  - Push event, pull request opened
- A workflow contains
  - One or more jobs
  - Each job has its own runner on which it is executed (typically a container)
    - Each job has one or more steps
    - Each step is either a script or command, or **another** Action



Continuous deployment with Github Actions

- Github Action specified in a YAML file
  - Configure a .github/workflows/[NAME].yml file
  - Define triggers (e.g., push, pull_request)
  - Specify jobs and steps (e.g., build, test)

```yaml
name: Build and test pipeline

# Controls when the action will run. Workflow runs when
manually triggered using the UI
# or API.
on: [push, pull_request]

# A workflow run is made up of one or more jobs that
can run sequentially or in parallel
jobs:
  build:
    runs-on: ubuntu-22.04
    steps:
      - uses: actions/checkout@v3
      - name: Set up JDK 21
        uses: actions/setup-java@v3
        with:
          java-version: 21
          distribution: 'temurin' # Use Temurin
distribution (default)
      - name: Verify Java version
        run: java -version
      - name: Build with Maven
        run: mvn package
      - name: Run JUnit tests
        run: mvn test
```

Trigger

Runner

Job

Steps

GitHub Actions demo

https://github.com/davsec-teaching/junit_demo